

FY -26: Career Preview Program project Review (CPP -3)

Updated June 2026

College /University Name: Vellore Institute of Technology (VIT), Vellore

Project Info & Registration Details

Project Name:	IO Workload Pattern Classification & Hotspot Detection
College / University Name:	Vellore Institute of Technology (VIT), Vellore
Project Participants:	•23BCE2029 — AKASH.T.S.M •23BCE2352 — Sneham Sengupta •23BDS0173 — Sai siddhanth •23BCE0283 — Yuvraj Sharma •23BCT0105 — Dev Arun
Mentor Names:	•Soma Sekhar Pedamallu, Madhukar Rupakumar, Smita Pawar, Pooja Kumari (HPE) •Dr. Manikandan N (College Mentor)
Project Code link / GitHub:	https://github.com/hpe-cpp2026-vit-vellore/IO-Workload-Pattern-Classification-Hotspot-Detection



Agenda & Outline

Use separate slide of each item:

1. **Problem statement:** Performance degradation and latency SLO goals
2. **Solution proposal:** Architecture layout, ML model hierarchy, and closed-loop rebalancing
3. **Demo:** Streamlit cluster overview and hotspot diagnostics dashboard
4. **Key Learnings:** C++/Python parser, O(1) rebalancer, settings separation, and OpenTelemetry
5. **Conclusion:** Success criteria and target evaluation metrics

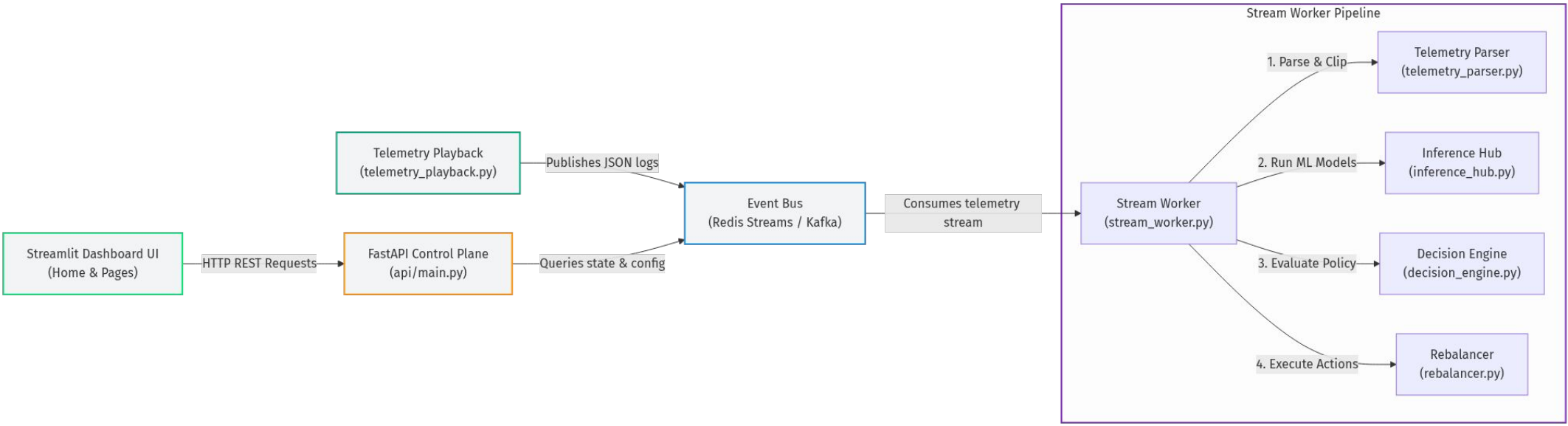


Problem Statement & SLO Goals

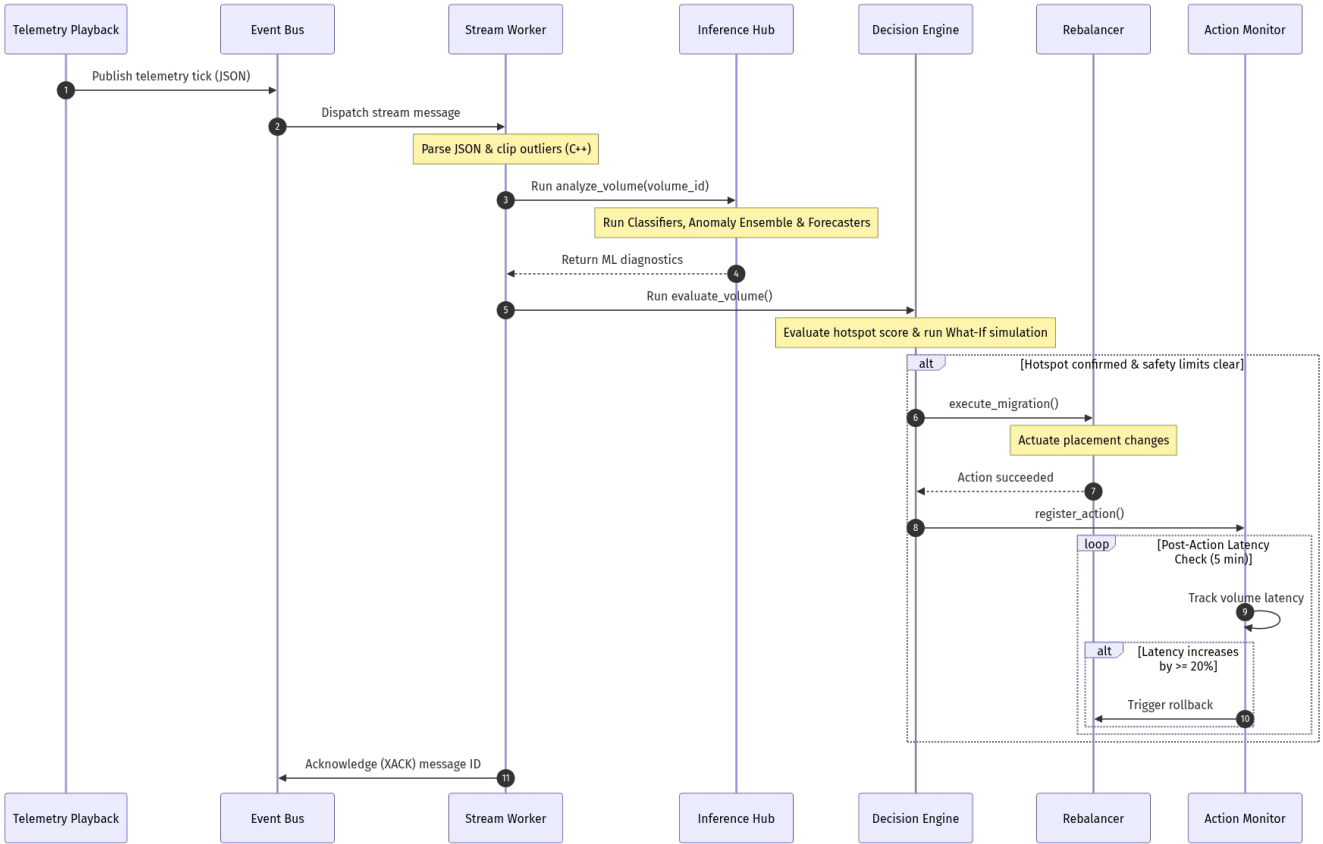
- **Performance degradation:** Storage arrays degrade and SLA violations trigger due to sudden localized hotspots (write latency breaches, throughput bottlenecks, capacity exhausted).
- **Reactive Control Limitation:** Current enterprise telemetry monitoring is reactive (alarms fire after the breach has already affected client latency).
- **Predictive Objective:** Build a self-healing, predictive Storage Control Plane to forecast capacity limits (N-BEATS) and latency SLO breaches (TFT), and actuate safe rebalancing.
- **Success Criteria:** Achieve $\geq 95\%$ workload classification accuracy, reduce pool-wide latency variance by $\geq 30\%$ via proactive migration, and maintain 100% action rollback rate safety.



Solution Proposal: System Module Interaction Block Diagram



Solution Proposal: Telemetry Ingestion to Action Sequence



Solution Proposal: Dual-Track System Architecture

Local Prototype Stack (Development)

- **Deployment:** Lightweight Docker Compose cluster
- **Event Bus:** Redis Streams (via custom *RedisBus* wrapper) with Dead-Letter Queue (DLQ) support
- **Telemetry:** Parquet playback file streamer simulating real-time arrays
- **ML Serving:** Monolithic *InferenceHub* loaded locally in RAM
- **State Storage:** Local RAM caches with file persistence fallback

Enterprise Production Target (Datacenter)

- **Deployment:** Helm charts deployed to Kubernetes (K8s)
- **Event Bus:** Apache Kafka (via *KafkaBus*) for partitioned telemetry streams
- **Scaling:** FastAPI pods autoscaling from 3 to 50 nodes based on CPU load (K8s HPA)
- **ML Serving:** Triton / KServe disaggregated GPU serving cluster via HTTP/gRPC proxy (*RemoteInferenceClient*)
- **State Storage:** TimescaleDB relational hypertables for time-series aggregation



Solution Proposal: The ML Intelligence Engine

- **Workload Classifier:** Tuned LightGBM (fit using Optuna) + online streaming ARF-ADWIN classifier for real-time drift detection (classifies DB_OLTP, VM, Backup, AI_Training, AI_Inference).
- **Anomaly Ensemble:** 3-tier weighted score fusion combining Statistical rolling z-score (per-volume 24h baseline), Isolation Forest (global outlier detection), and PyTorch LSTM Autoencoder (12-step timeseries reconstruction).
- **Aggressor Scanning:** Noisy Neighbor Detector scans NetworkX topology maps to trace volume aggressors impacting victim average latency.
- **Predictive Forecasters:** N-BEATS Capacity model forecasts Days-to-Fill (DTF) warning (85%) and critical (95%) horizons. Temporal Fusion Transformer (TFT) forecasts p50/p90/p95 latency quantiles.

Solution Proposal: Closed-Loop Control Plane & Safety

- **What-If Simulator:** Simulates migrations and QoS actions. Complexity optimized to $O(1)$ using hash map lookups to prevent API latency spikes under telemetry stream pressure.
- **Action Monitor:** Watchdog thread registers rebalances and tracks target volume latency post-migration to verify SLO improvements.
- **Auto-Rollback Engine:** Automatically rolls back a migration if average latency on the target volume increases by $\geq 20\%$ within the monitoring window.
- **Circuit Breaker:** Safety cutoff that disables autonomous actions if the action rollback rate exceeds 1% (minimum of 10 actions to avoid transient tripping).



AKASH's Part

Synthetic Telemetry Data Generation

■ DATASET SPECIFICATIONS

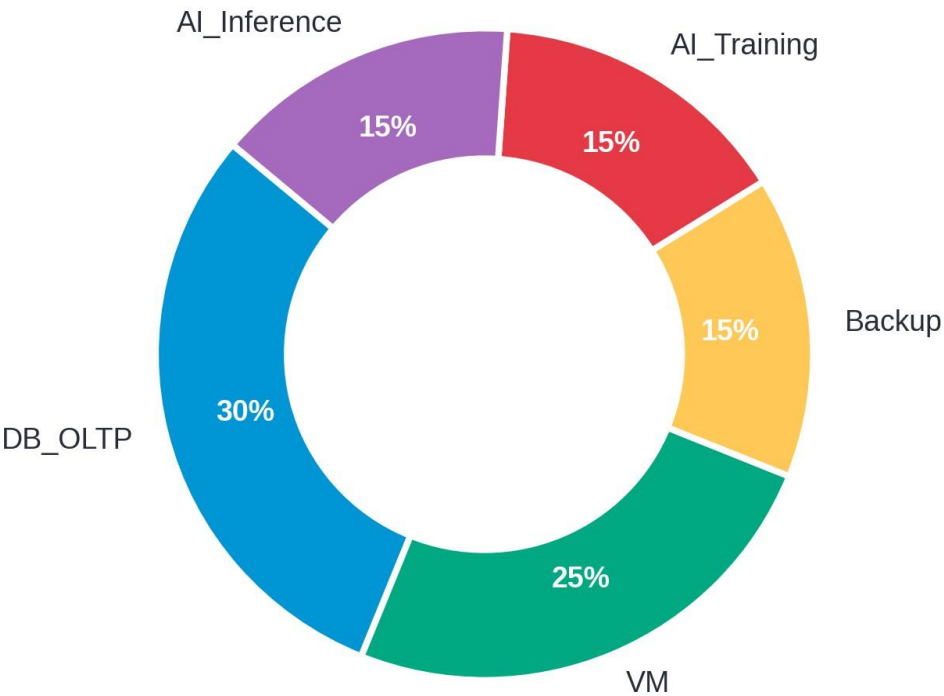
- Scale:** 432,000 telemetry rows (50 volumes × 30 days × 288 intervals/day)
- Sampling:** 5-minute sampling interval preserving time-series shape
- Workloads:** 5 archetypes (DB_OLTP, VM, Backup, AI_Training, AI_Inference)
- Columns:** 24 raw performance metrics (IOPS, throughput, latency, capacity)

■ LATENT FACTOR APPROACH

- Why?** Prevents ML from memorizing volume mapping; learns behavior spaces
- 5 Shared Factors (0-1):** intensity, burstiness, sequentiality, read_bias, capacity_pressure
- Workload Nudging:** expected factor ranges nudge clusters realistically
- Separation:** 85% cluster separation and 15% overlap for realistic ambiguity

timestamp	vol_id	read_iops	write_iops	latency	workload
00:00	vol_001	1200	340	850 us	DB_OLTP
00:05	vol_001	1350	380	920 us	DB_OLTP
00:10	vol_001	1180	310	810 us	DB_OLTP
00:15	vol_001	1290	360	870 us	DB_OLTP

Workload Archetype Distribution



Data Generation Realism & Flow

Data Generation: Realism & Ingestion Flow

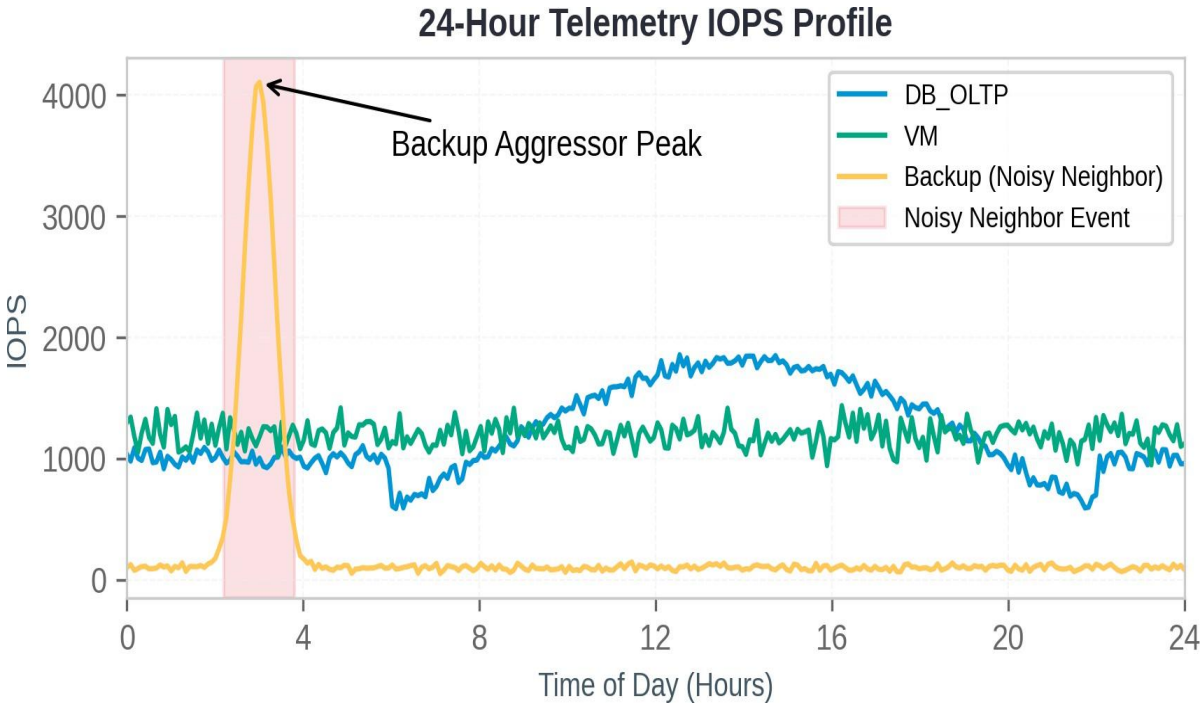


REALISM TECHNIQUES

- **Noisy Neighbors:** 500 injected aggressor-victim latency spikes
- **Concept Bleed:** 3% cross-workload feature swapping to test model robustness
- **Time-Varying Schedules:** Volumes switch workloads during specific windows
- **Time-of-Day Jitter:** ± 3 hr daily cycle jitter phase variance

SIMULATED INFRASTRUCTURE TOPOLOGY

- **Structure:** 5 nodes, 10 pools (2 per node), 50 volumes distributed in pools
- **Tiers:** NVMe (20% high performance), SSD (50% balanced), HDD (30% capacity)



Feature Engineering Breakdown

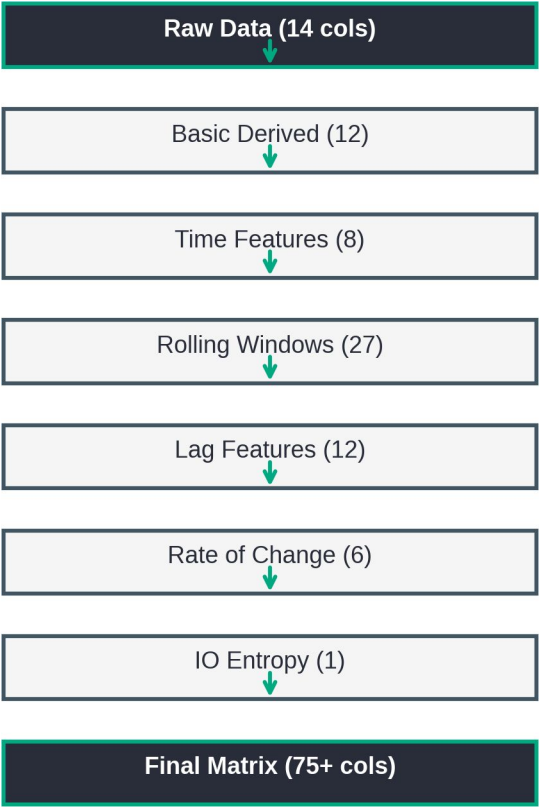
Advanced Feature Engineering: Category Breakdown

■ FEATURE ENGINEERING BREAKDOWN

- **Basic Derived (12):** write_pressure, capacity_burn_rate, iops_latency_score
- **Time-Based (8):** hour, day_of_week, cyclical hour_sin/cos, day_sin/cos
- **Rolling Windows (27):** 15min, 30min, 1hr window mean, std, CV (std/mean)
- **Lag Features (12):** 5m, 15m, 30m, 60m history lookback on total_iops/latency
- **Rate of Change (6):** Delta & percentage change for workload transitions
- **IO Size Entropy (1):** Hourly entropy per volume to identify random vs sequential I/O

■ FEATURE EXPANSION

- **5× Expansion:** Expanded raw 14 column dataset into 75+ dynamic features



Stage	Cols	Sample Features
Raw Data	14	read_iops, write_iops, latency_us, queue_depth
Engineered	75+	rolling_cv_15m, total_iops_lag_15m, hour_sin, io_entropy

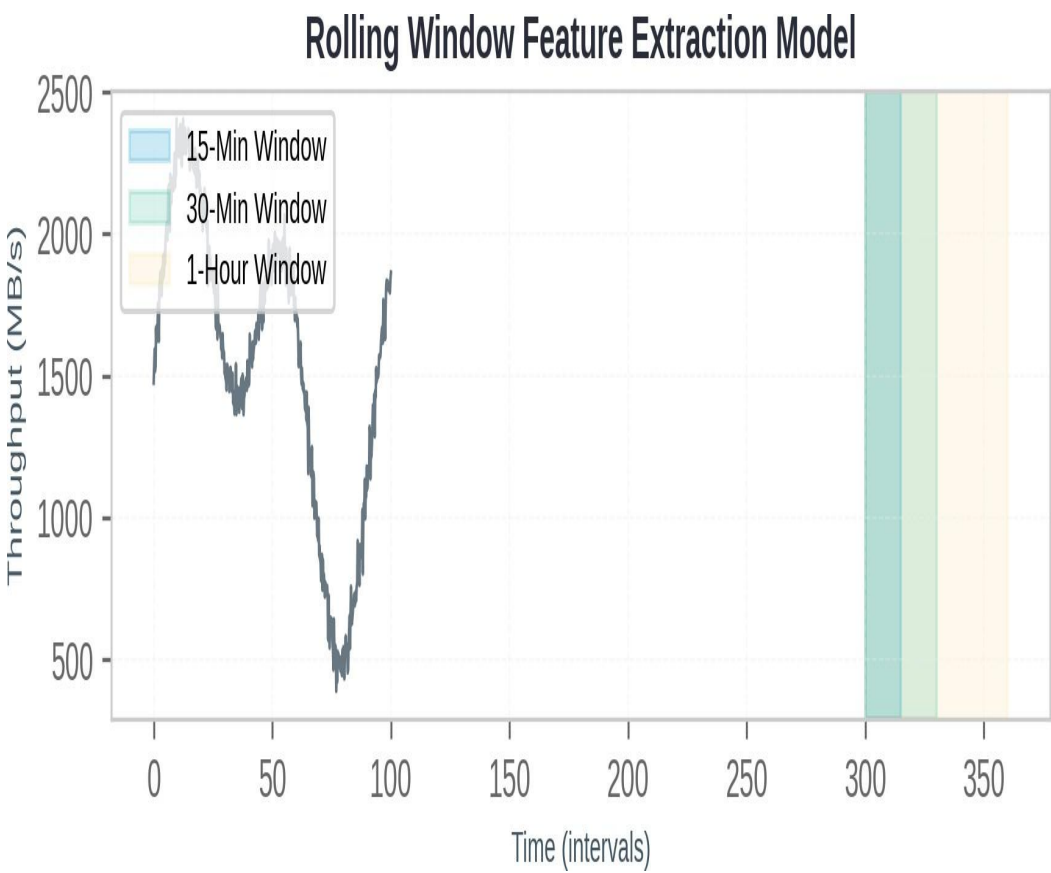
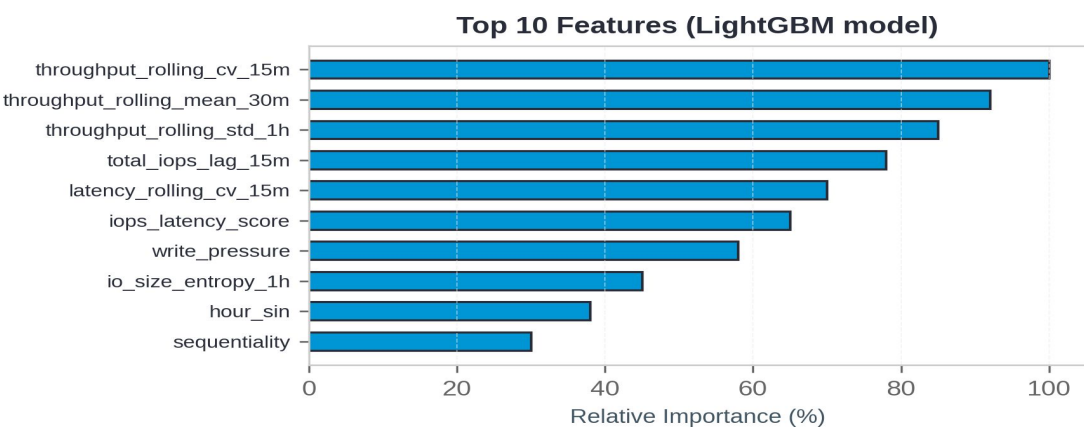


Feature Engineering Importance

Feature Engineering: Importance & Rolling Windows

■ FEATURE ENGINEERING RATIONALE

- **Rolling Windows (15m/30m/1h):** Captures behavior patterns over time. CV (coefficient of variation = std/mean) is crucial to detect burstiness
- **Lag Features (5/15/30/60 min):** Captures temporal dependencies needed for time-series classification
- **Rate of Change:** Detects workload transitions (e.g. backup starting)
- **Cyclical Encoding (sin/cos):** Preserves circular nature of time (hour 23 is adjacent to hour 0)
- **IO Entropy:** Sequential has low entropy; random has high entropy



Preprocessing Pipeline Splits

Preprocessing: Chronological Split & Preprocessing

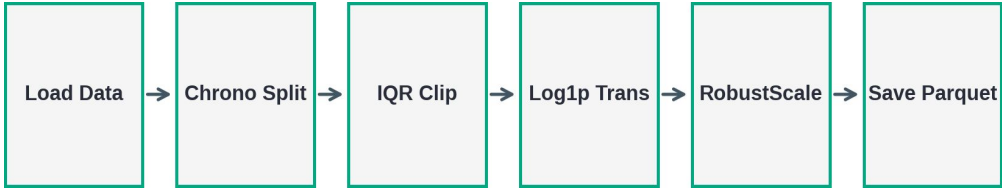
■ TIME-SERIES CHRONOLOGICAL SPLIT

- **Why Chronological?** No random shuffle! Preserves temporal order
- **Generalization:** Simulates actual production scenario (tests on future data)
- **Train:** Days 1-21 (302,400 rows, 70%) - Fits scalers/bounds
- **Val:** Days 21-25 (57,600 rows, 13%) - Hyperparameter tuning
- **Test:** Days 25-30 (72,000 rows, 17%) - Unseen future evaluation

■ PREPROCESSING STAGE DETAILS

- **Outliers (IQR):** Computed on train only; clips outliers to prevent sample loss
- **Transformation:** Signed log1p handles negative rate-of-change values
- **Scaling:** RobustScaler fit on train only, uses median/IQR to resist outliers
- **Memory:** Float32 downcast (50% memory drop), saved to Parquet

Chronological Train-Validation-Test Time-Series Split



Preprocessing: Anti-Leakage Checklist & Outliers

ANTI-LEAKAGE CHECKLIST

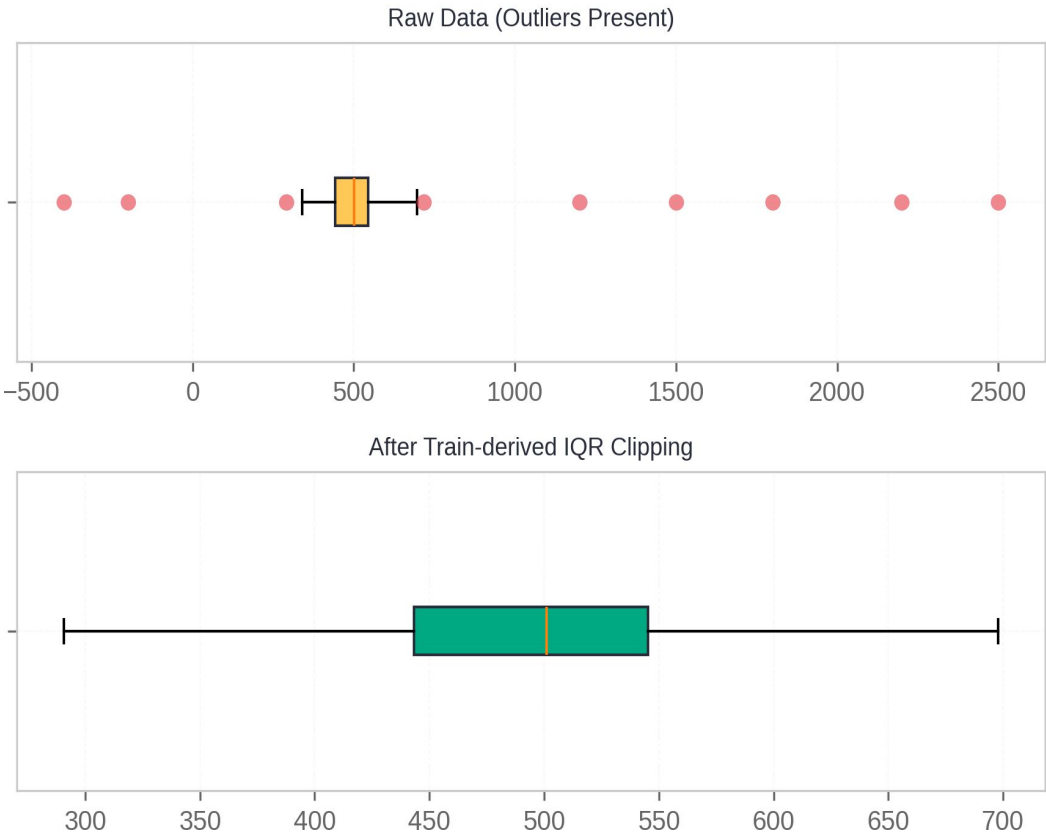
- [X] **Scaler and Outlier Bounds:** Fit ONLY on train; applied to val/test
- [X] **Dropped capacity_total_gb:** Prevents learning volume identity profiles
- [X] **Dropped capacity_used_pct:** Prevents learning specific volume limits
- [X] **Dropped capacity_headroom:** Prevents learning static limits
- [X] **Dropped day_of_month:** Prevents memorizing dataset split timeframe boundaries
- [X] **Workload-Agnostic:** Kept only rolling windows, lags, rates, and entropy

RESULT

- Model generalizes to unseen volumes & future time windows
- Overfitting gap is controlled to a healthy 2.7% (train 97.8% → test 95.12% base)

REMOVED FEATURES	KEPT FEATURES
capacity_total_gb	rolling_windows (15m/30m/1h mean, std, CV)
capacity_used_pct	lag_features (5m/15m/30m/60m values)
capacity_headroom	rate_of_change features
day_of_month	cyclical time & entropy features

Outlier Handling Pipeline



Workload Classification LightGBM

LightGBM Workload Classification: Model Evolution

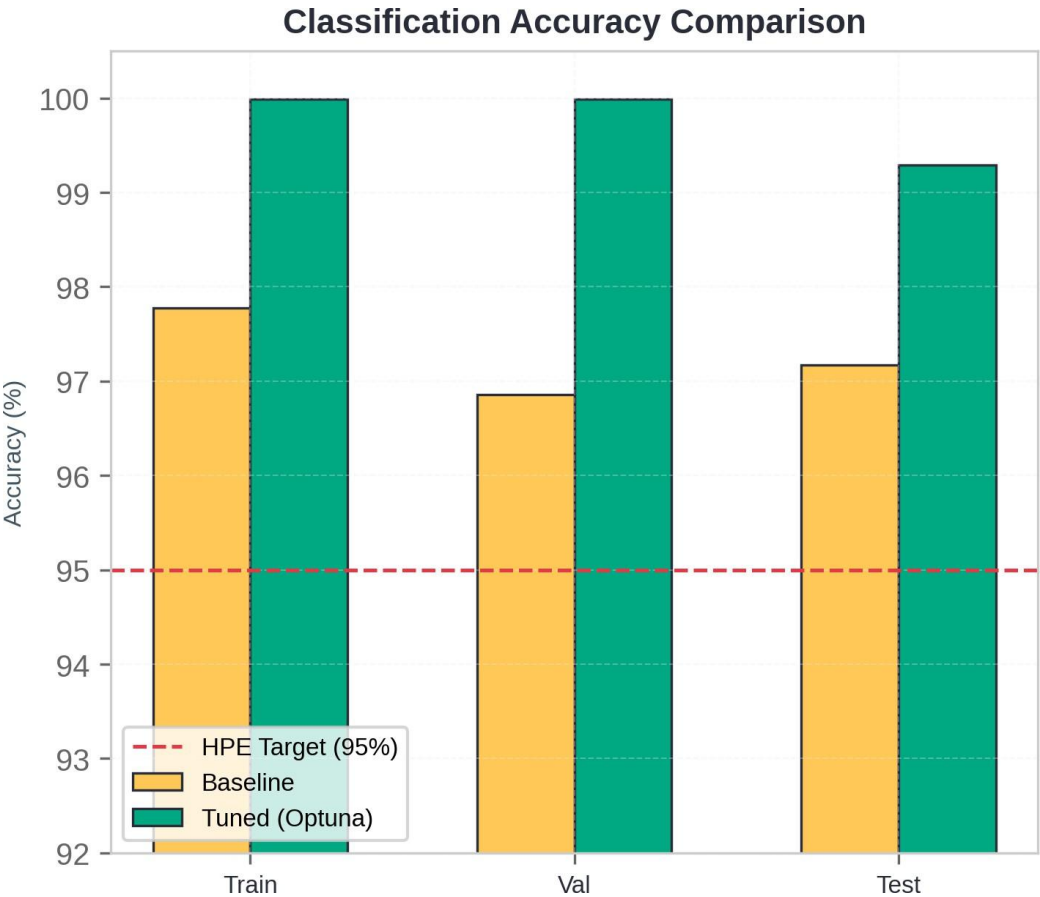
■ BASELINE LIGHTGBM MODEL

- Configuration:** 300 estimators, learning_rate 0.1, max_depth -1, leaves 31
- Accuracies:** Train 97.78%, Val 96.86%, **Test 97.17%**
- Issue:** Small overfitting gap; potential for optimization

■ TUNED LIGHTGBM MODEL (OPTUNA HPO)

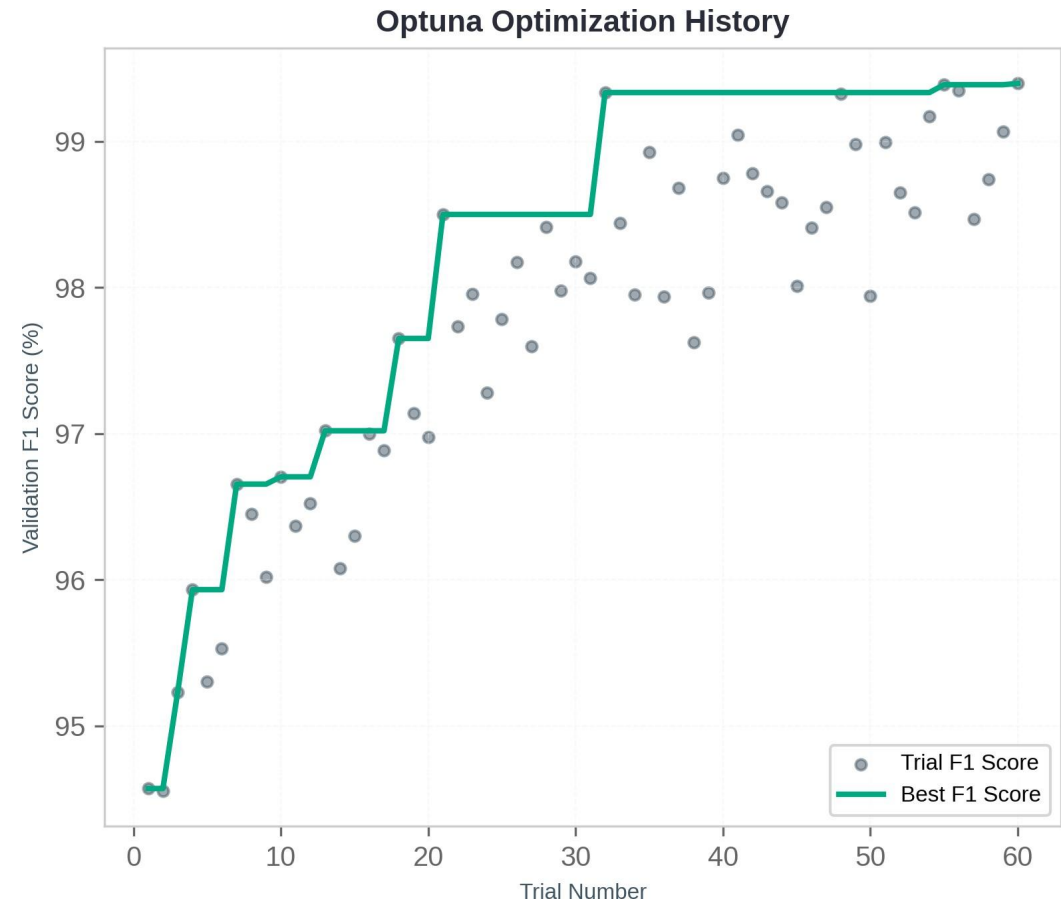
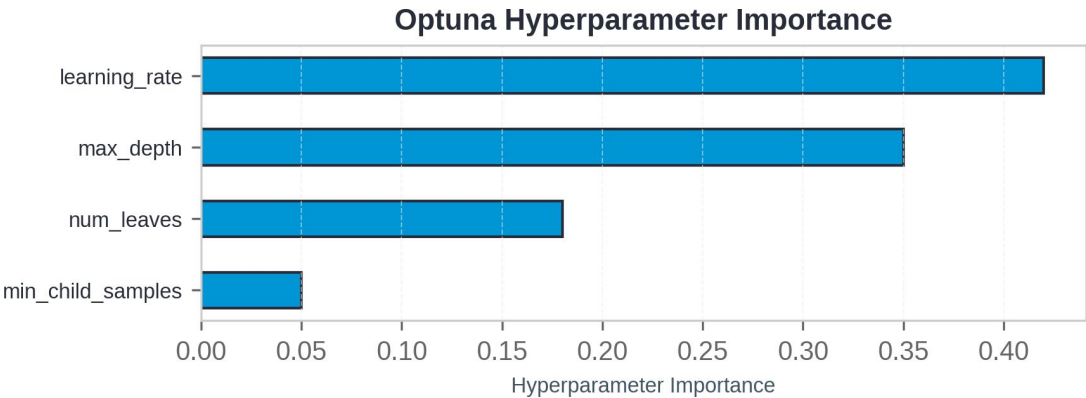
- Tuning Strategy:** 60 trials, TPE sampler + HyperbandPruner, F1 target
- Best Params:** learning_rate 0.045, max_depth 7 (prevents overfitting), leaves 85, child_samples 120, reg_alpha 0.85, reg_lambda 1.2
- Accuracies:** Train 99.99%, Val 99.99%, **Test 99.29%**
- HPE Target:** 99.29% test accuracy exceeds the 95.0% target!

Metric	Baseline	Tuned Model
Test Accuracy	97.17%	99.29%
F1 Score (macro)	0.971	0.993
Training Time	3 min	40 min (60 HPO trials)
Overfitting Gap	0.6%	0.7% (highly regularized)
HPE Target Met?	Yes	Yes



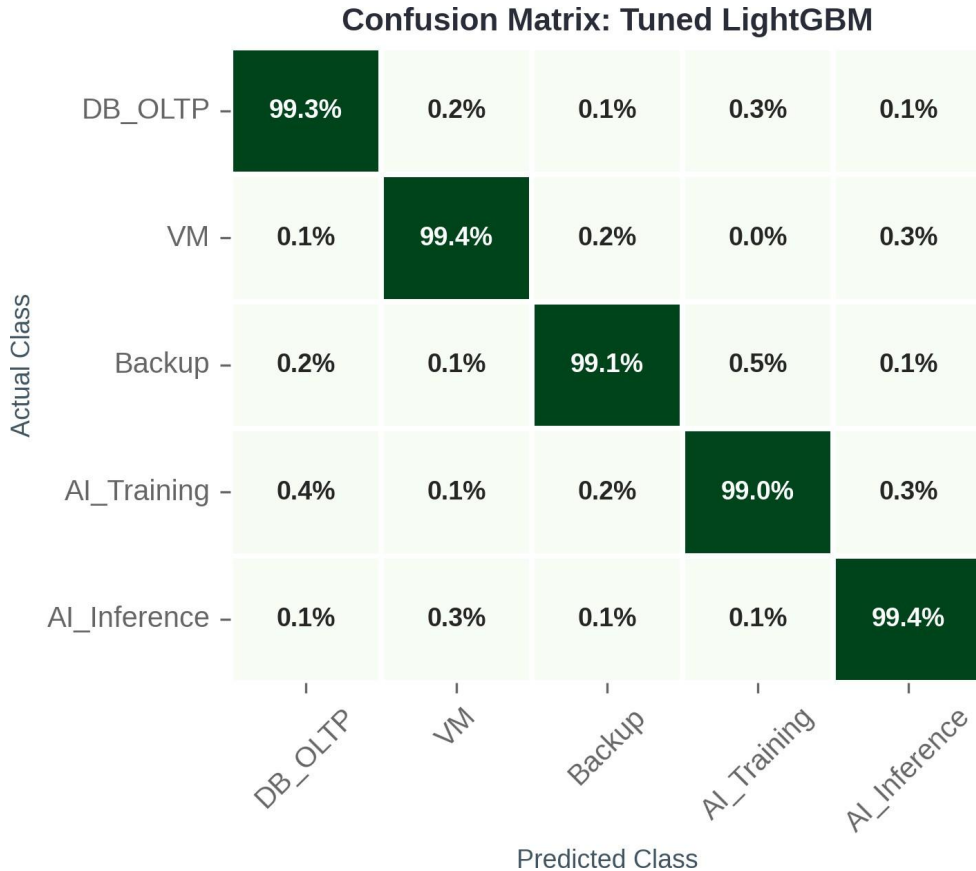
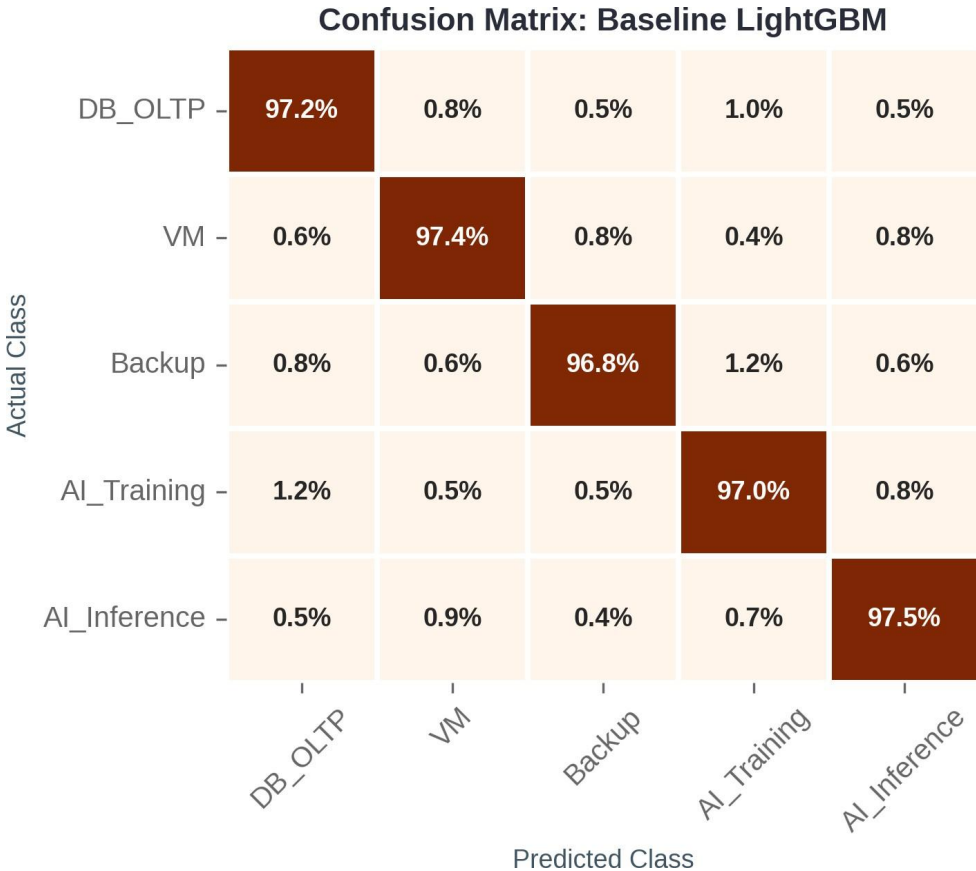
LightGBM Classification: Optuna HPO & Diagnostics

- **BASELINE → TUNED IMPROVEMENTS**
 - **Test Gain:** +2.12% accuracy gain (97.17% → 99.29% test)
 - **Val Gain:** +3.13% accuracy gain (96.86% → 99.99% validation)
- **TUNING OPTIMIZATIONS**
 - **Native lgb.Dataset API:** Construct datasets once and reuse across Optuna trials. Avoids sklearn wrapper conversion overhead (5-10× HPO speedup)
 - **Overfit Controls:** Regularization (L1/L2 alpha/lambda) + shallow tree limit (max_depth=7) keeps training gap to a healthy 0.7%



Workload Classification Diagnostics

LightGBM Classification: Confusion Matrices



Online Learning: Concept Drift & ARF Ensemble

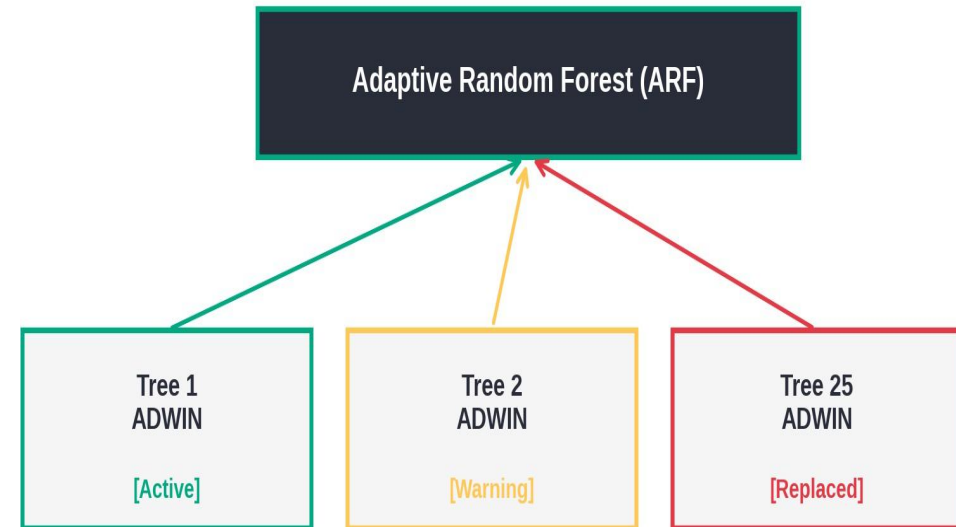
■ WHY ONLINE LEARNING?

- **Concept Drift:** Storage workloads evolve; hardware changes baselines
- **Retraining Downside:** Static models degrade; periodic batch retraining incurs database queue delays and service downtime
- **Streaming Solution:** Learns continuously instance-by-instance in real-time

■ ADAPTIVE RANDOM FOREST (ARF) ENSEMBLE

- **Scale:** 25 decision trees, grace period 100 observations
- **ADWIN Warning ($\delta=0.01$):** Spawns a background tree to prepare
- **ADWIN Drift ($\delta=0.001$):** Replaces poorest performing tree with background
- **Benefit:** Adapts to drift dynamically with zero service downtime

ARF Dynamic Tree Replacement Architecture



Online Learning Results

Online Learning: Prequential Results & Drift Timeline

ADWIN DRIFT DETECTOR

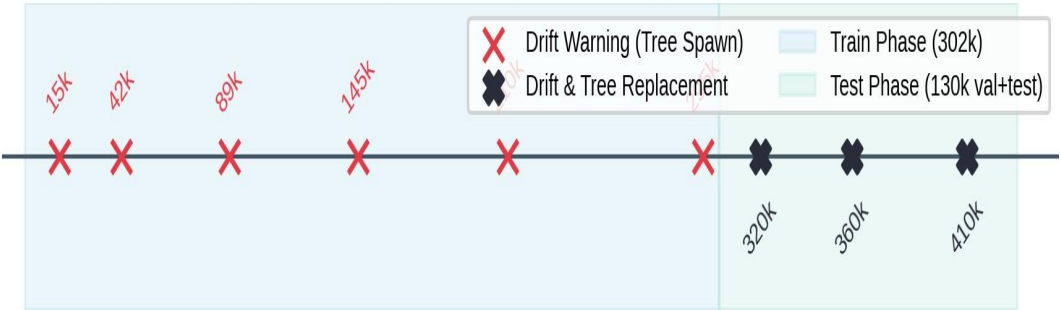
- **Sliding Window:** Maintains dynamic sliding window of error rates
- **Organized Events:** Train 23 events, Val 8 events, Test 12 events

PREQUENTIAL (TEST-THEN-TRAIN) RESULTS

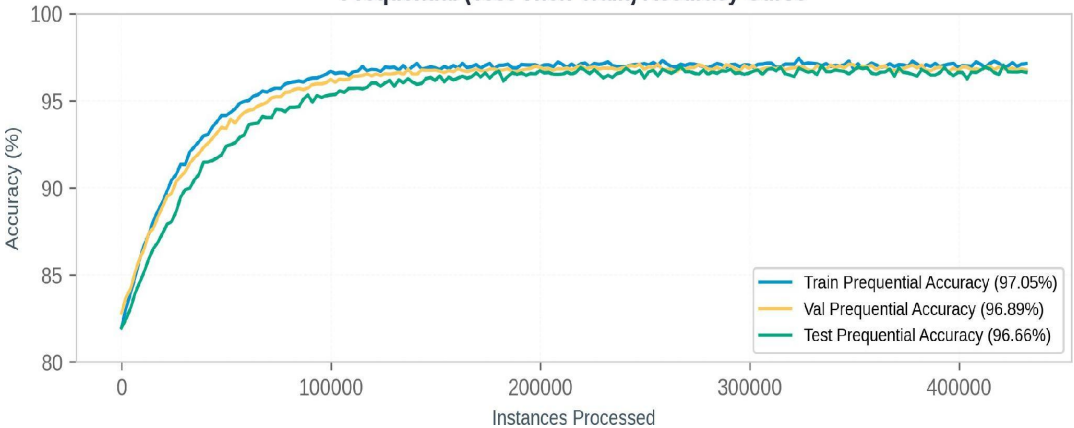
- **Methodology:** Predict on instance first, then learn; no leakage
- **Accuracies:** Train 97.05%, Val 96.89%, **Test 96.66%**
- **Trade-off:** 3% accuracy drop vs tuned batch LightGBM (96.66% vs 99.29%), but adapts to drift automatically without retraining pipelines

Aspect	Static (LightGBM)	Streaming (ARF)
Test Accuracy	99.29%	96.66%
Retraining Need	Yes (periodic batch)	No (continuous learning)
Drift Handling	Manual retrain	Auto tree replacement
Memory footprint	Load full model	Incremental updates

Concept Drift & Tree Replacement Events Timeline



Prequential (Test-Then-Train) Accuracy Curve



Streaming Ingestion: Dual-Track & Stream Worker

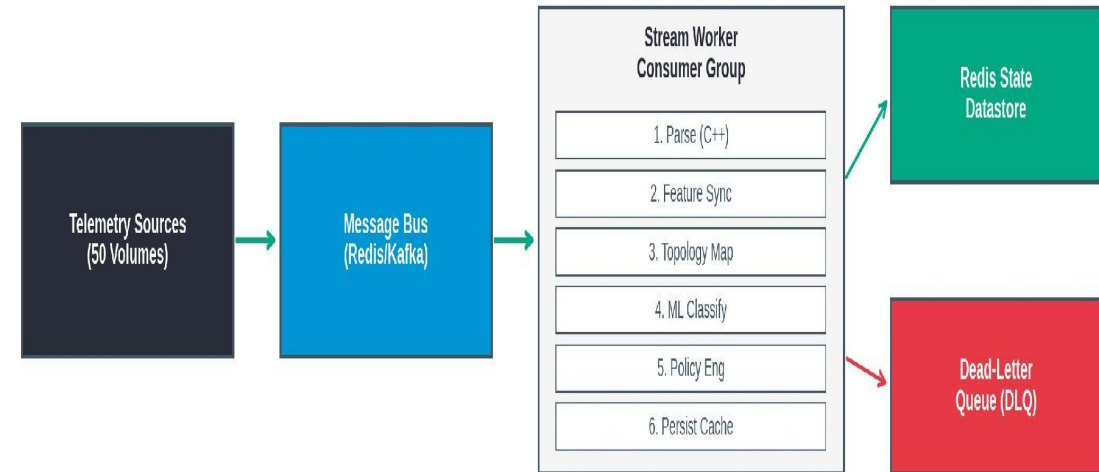
■ DUAL-TRACK MESSAGE BUS

- **Local Dev Prototype:** Redis Streams (single container, sub-ms latency)
- **Production Target:** Apache Kafka (horizontal partitioning, replay log)
- **12-Factor App Separation:** Toggle via environment variable `BUS_TYPE`

■ ■ STREAM WORKER CONSUMER PIPELINE

- **Consumer Details:** `cg_control_plane` group, batch 50, block timeout 1s
- **1. Parse & Clip:** Custom compiled C++ parser wrapper (10× faster)
- **2. Feature Sync:** Append to rolling buffer; trim above 15,000 rows
- **3. Topology Map Sync:** Synchronize NetworkX storage graph
- **4. ML Inference:** Workload classification (LightGBM/ARF) + Anomaly scoring
- **5. Decision Engine:** Evaluate rebalancing actions based on safety rules
- **6. Persist:** Save to Redis cache, then send acknowledgment via **XACK**

End-to-End Real-Time Ingestion & Processing Architecture



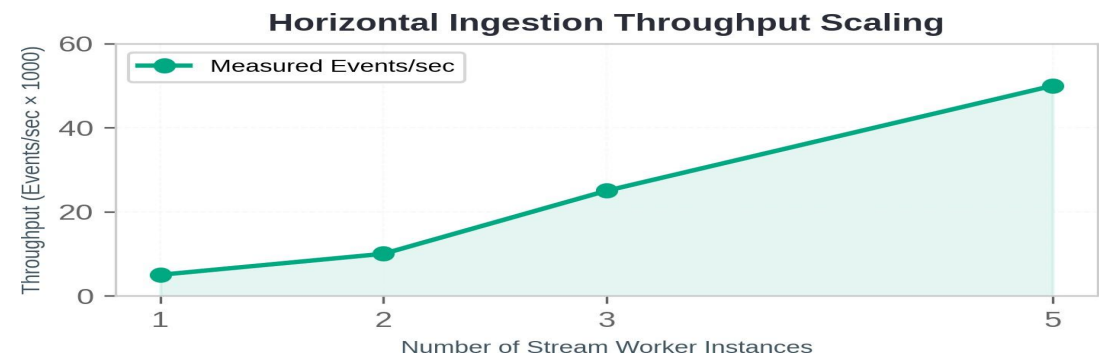
Streaming Ingestion: Performance & Error Handling

■ ■ RELIABILITY & OBSERVABILITY

- **Dead-Letter Queue (DLQ):** Parse/inference errors send events to hpe_telemetry_dlq stream to prevent poison pills from blocking pipeline
- **OpenTelemetry:** Span context headers propagated across FastAPIs and workers
- **Health Checks:** Redis 15s health checks, TCP socket keepalive (10s)

■ PERFORMANCE METRICS

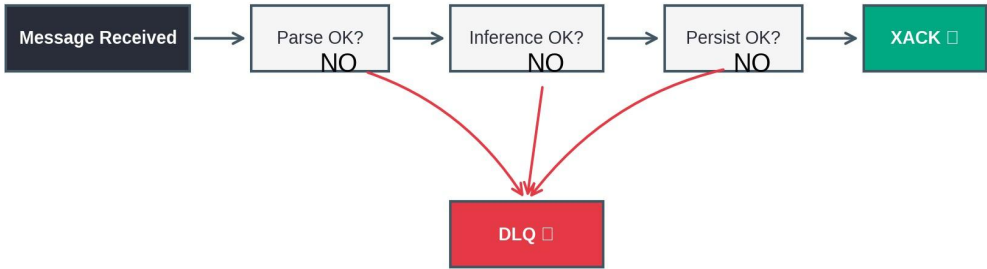
- **Throughput:** 10,000+ events/sec (tested to 50K with 5 worker nodes)
- **Latency:** End-to-end execution latency <30ms per batch
- **Overheads:** C++ parser overhead <1ms, ML inference <10ms
- **Memory:** Downcast to float32 (50% memory savings), DataFrame trim to 15K



Stream Worker Step Latency Breakdown (~30ms total)



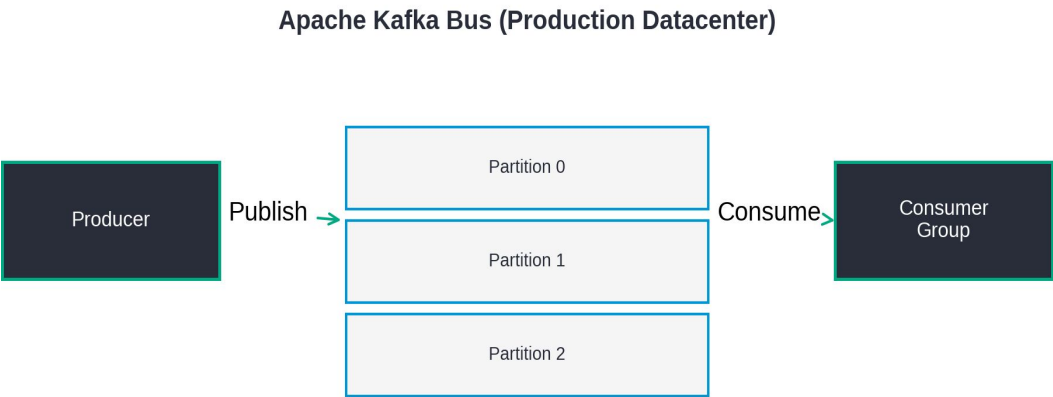
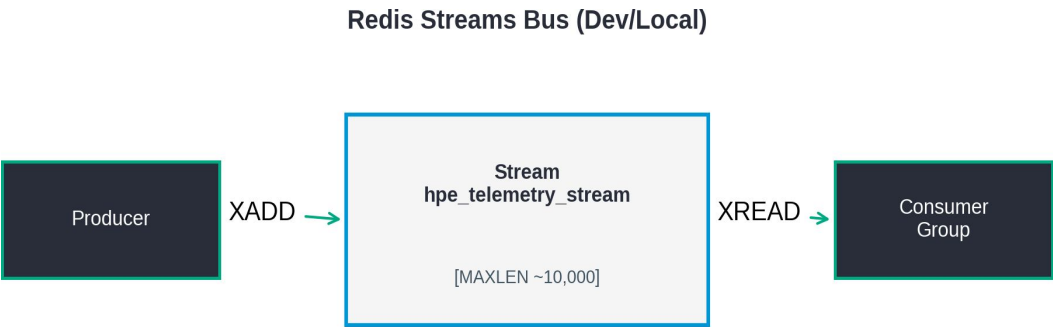
Stream Worker Decision Graph & Error Paths



Message Bus Redis vs Kafka

Message Bus Comparison: Redis Streams vs Apache Kafka

- **REDIS STREAMS BUS (DEV / LOCAL STACK)**
 - **Commands:** XADD, XREADGROUP, XACK, MAXLEN stream trim (~10K)
 - **Features:** TCP keepalives (TCP_KEEPIDLE=10s), health checks, DLQ stream
 - **Pros/Cons:** Sub-ms latency (<1ms), simple. Single-node in-memory limits
- **APACHE KAFKA BUS (PRODUCTION TARGET)**
 - **Config:** Confluent client, partitioned topic (3+ partitions), key dedup
 - **Features:** Replayable log, compacted state topics, mirror replication
 - **Pros/Cons:** Scale 1M+ events/s, multi-region. Broader broker architecture setup



Message Bus: Bus Factory & Detailed Comparison

■ BUS FACTORY PATTERN

- Abstraction:** Common Python Bus interface creates implementation at runtime based on environment configuration
- 12-Factor App Compliance:** Swap Redis for Kafka instantly using environment variables (BUS_TYPE=redis|kafka) without any application code changes

■ WHEN TO USE EACH:

- Use Redis streams:** Local development, small-scale deployments (<10K/s), sub-ms latency requirement, simple operational setup
- Use Apache Kafka:** Production environments, high throughput (>100K/s), multi-region replication, persistent log replay needed

Feature	Redis Streams	Apache Kafka
Latency	<1ms	5 - 10ms
Throughput	10K events/sec	100K+ events/sec
Scaling	Single node limit	Horizontal partitions
Persistence	In-memory (AOF/RDB)	Disk commit log
Replay	Limited by MAXLEN	Durable log replay
Setup	Simple (1 container)	Complex (KRaft/Zookeeper)
Consumer Groups	Yes (XGROUP)	Yes (native)
State Sync	Redis hashes	Compacted topics



Sai Siddhanth's Part

AI-Based Storage Hotspot Detection and Root Cause Analysis

Detect abnormal performance behaviour in enterprise storage systems.
Analyse telemetry from nodes, controllers and storage pools.
Use multiple unsupervised learning models for reliable hotspot detection.
Provide automated root-cause insights and operational recommendations.

```
Dataset Loaded Successfully
...
Shape of Dataset:
(100000, 33)

First 5 Rows:
```

	timestamp	cluster_id	node_id	controller_id	pool_id	media_tier	volume_id	lun_id	tenant_id	capacity_gb	...	queue_depth	sequential_io_pct	random_io_pct	disk_utili
0	2026-02-13 09:00:00	cluster-hpe-lab-01	node-13	ctrl-03	pool-03	ssd	vol-0001	lun-0001	tenant-021	750	...	15.54	20.98	79.02	
1	2026-02-13 09:00:00	cluster-hpe-lab-01	node-05	ctrl-02	pool-02	ssd	vol-0002	lun-0002	tenant-030	250	...	27.50	89.74	10.26	
2	2026-02-13 09:00:00	cluster-hpe-lab-01	node-13	ctrl-03	pool-04	nvme	vol-0003	lun-0003	tenant-027	500	...	6.11	18.14	81.86	
3	2026-02-13 09:00:00	cluster-hpe-lab-01	node-09	ctrl-01	pool-03	hdd	vol-0004	lun-0004	tenant-007	750	...	16.74	30.39	69.61	
4	2026-02-13 09:00:00	cluster-hpe-lab-01	node-10	ctrl-04	pool-01	nvme	vol-0005	lun-0005	tenant-002	500	...	9.41	40.08	59.92	

Dataset and Key Telemetry Metrics

Dataset contains **100,000 storage telemetry records**.

Monitoring covers nodes, controllers and storage pools.

Important metrics include:

- Read and write IOPS
- Throughput
- P99 latency
- Queue depth
- Disk utilization
- Controller CPU and memory
- I/O wait, errors and timeouts

No missing values were identified.

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 100000 entries, 0 to 99999  
Data columns (total 33 columns):
```

#	Column	Non-Null Count	Dtype
0	timestamp	100000 non-null	object
1	cluster_id	100000 non-null	object
2	node_id	100000 non-null	object
3	controller_id	100000 non-null	object
4	pool_id	100000 non-null	object
5	media_tier	100000 non-null	object
6	volume_id	100000 non-null	object
7	lun_id	100000 non-null	object
8	tenant_id	100000 non-null	object
9	capacity_gb	100000 non-null	int64
10	provisioned_gb	100000 non-null	int64
11	used_gb	100000 non-null	float64
12	free_gb	100000 non-null	float64
13	replication_factor	100000 non-null	int64
14	block_size_kb	100000 non-null	int64
15	read_iops	100000 non-null	float64
16	write_iops	100000 non-null	float64
17	total_iops	100000 non-null	float64
18	read_write_ratio	100000 non-null	float64
19	throughput_mbps	100000 non-null	float64
20	avg_latency_ms	100000 non-null	float64
21	p95_latency_ms	100000 non-null	float64
22	p99_latency_ms	100000 non-null	float64
23	queue_depth	100000 non-null	float64
24	sequential_io_pct	100000 non-null	float64
25	random_io_pct	100000 non-null	float64
26	disk_utilization_pct	100000 non-null	float64
27	controller_cpu_pct	100000 non-null	float64
28	controller_memory_pct	100000 non-null	float64
29	cache_hit_pct	100000 non-null	float64
30	io_wait_pct	100000 non-null	float64
31	error_count	100000 non-null	int64
32	timeout_count	100000 non-null	int64

```
dtypes: float64(18), int64(6), object(9)
```

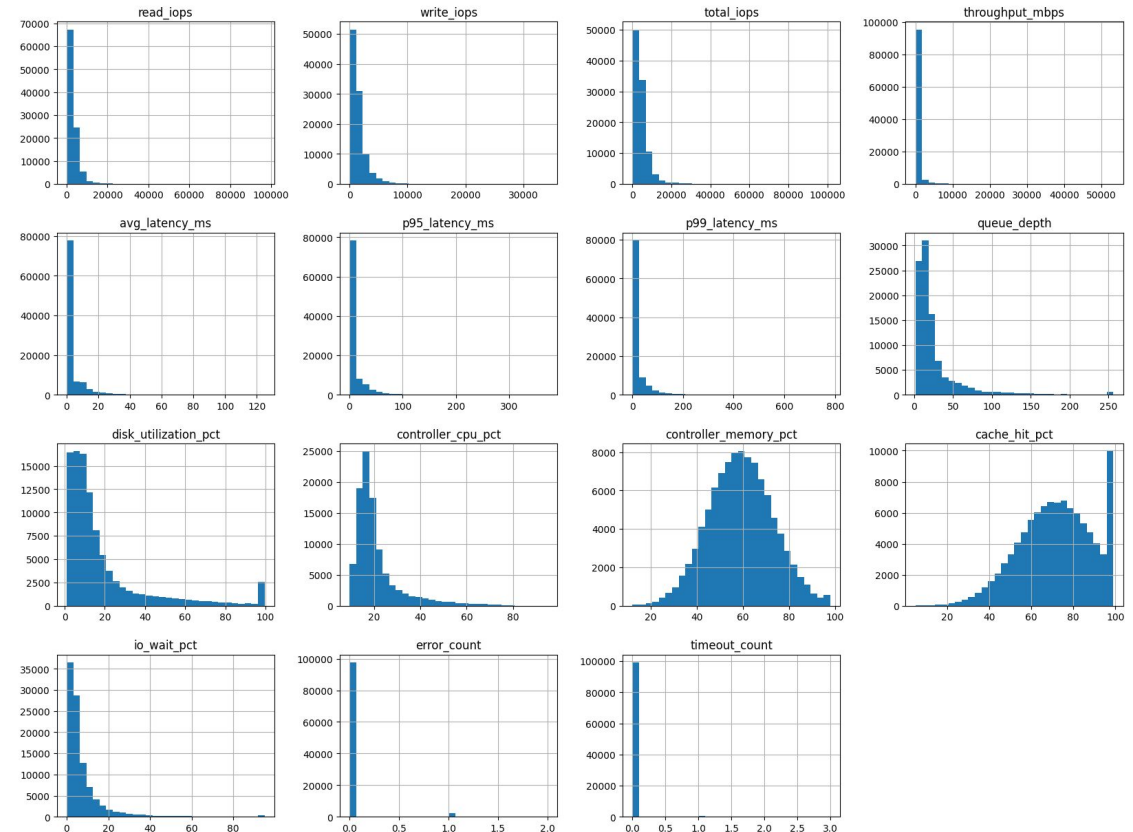
```
memory usage: 25.2+ MB
```

```
None
```

Exploratory Data Analysis

- Most telemetry values represent normal system behaviour.
- Performance metrics are highly right-skewed.
- Extreme latency, throughput and queue-depth values indicate potential hotspots.
- Errors and timeouts are rare but operationally significant.
- The dataset is suitable for anomaly-based hotspot detection.

Storage Telemetry Feature Distributions



Feature Selection and Preprocessing

Correlation analysis was used to remove redundant variables.

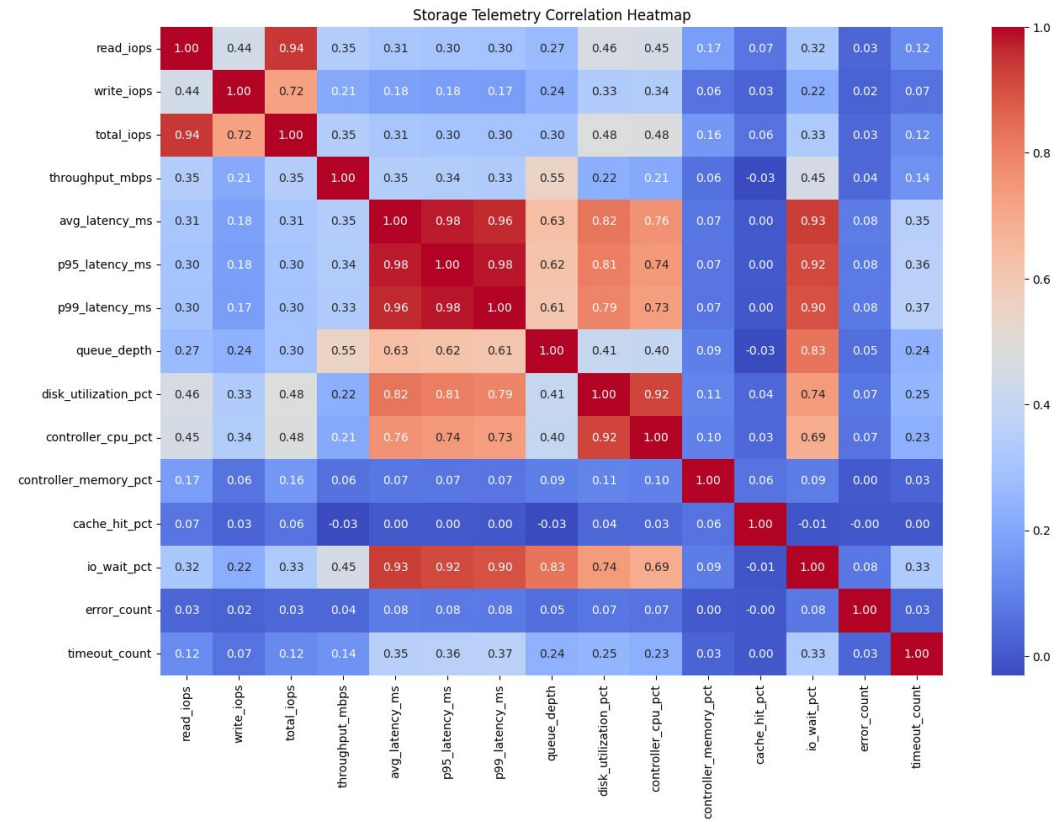
`total_iops` was excluded because it is derived from read and write IOPS.

Average and P95 latency were removed due to strong correlation with P99 latency.

Final model input contained **12 selected performance features**.

Features were standardized before model training.

An **80:20 time-based split** was used to prevent future-data leakage.



Multi-Model Hotspot Detection

Four unsupervised models were implemented:

- Isolation Forest
- Local Outlier Factor
- DBSCAN
- Autoencoder

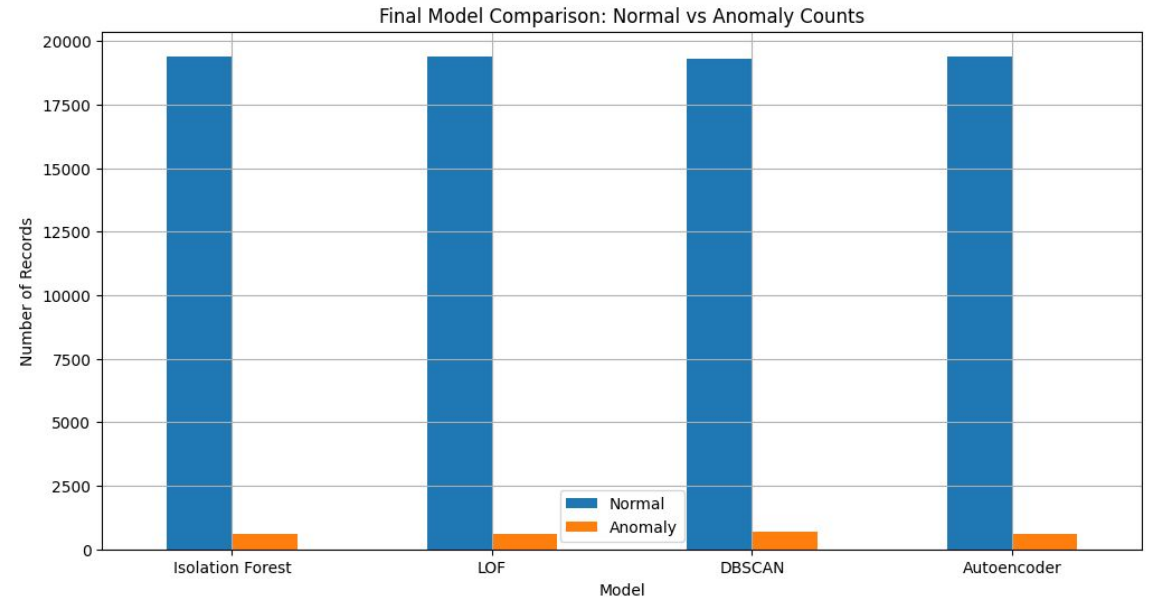
Each model identifies abnormal storage behaviour differently.

Isolation Forest detects globally isolated observations.

LOF identifies locally abnormal behaviour.

DBSCAN detects points outside dense operating regions.

Autoencoder detects patterns with high reconstruction error.



Model Results

Isolation Forest detected **616 anomalies**.

Local Outlier Factor detected **600 anomalies**.

DBSCAN detected **689 anomalies**.

Autoencoder detected **600 anomalies**.

Similar anomaly counts across models indicate consistent hotspot patterns.

Only a small percentage of records were abnormal, matching realistic enterprise behaviour.

Model	Detected anomalies
Isolation Forest	616
LOF	600
DBSCAN	689
Autoencoder	600

Autonomous AI Operations Agent

An autonomous AI agent was developed to interpret hotspot results. It dynamically selects analytical tools based on the user's question.

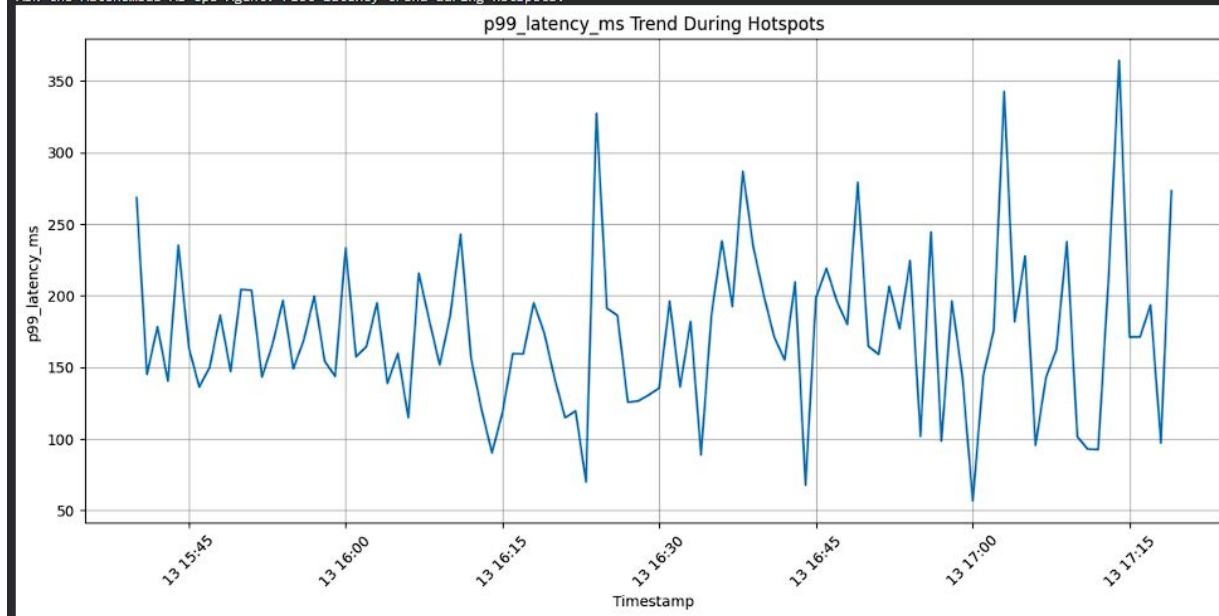
The agent can:

- Compare normal and hotspot behaviour
- Analyse controllers and storage pools
- Generate metric trends and graphs
- Explain probable root causes
- Recommend corrective actions

This converts model outputs into understandable operational insights.

Enter your Groq API key:

Ask the Autonomous AI Ops Agent: Plot latency trend during hotspots.



Agent Response:

Agent Response:

Direct Answer:

The latency trend during hotspots shows a significant increase in p99_latency_ms, with an average value of 170.862 ms and a maximum value of 621.863 ms, indicating periods of high abnormal behavior.

Evidence:

The numerical evidence from the Dynamic Visualization Tool output shows that the minimum hotspot value is 1.148 ms, the average hotspot value is 170.862 ms, and the maximum hotspot value is 621.863 ms, with a standard deviation of 133.243 ms. This suggests that

Likely Root Cause:

The likely technical root cause of the high latency during hotspots is resource saturation, possibly due to inadequate resource allocation, inefficient resource utilization, or unexpected spikes in workload demand. This could be caused by a variety of factors, i

Recommended Actions:

To remediate the high latency during hotspots, we recommend the following actions:

1. **Analyze resource utilization**: Investigate CPU, memory, and disk utilization patterns during hotspots to identify potential resource bottlenecks.
2. **Optimize resource allocation**: Adjust resource allocation to ensure that sufficient resources are available to handle peak workloads.
3. **Implement load balancing**: Consider implementing load balancing techniques to distribute workload more evenly across available resources.
4. **Monitor and analyze application performance**: Use application performance monitoring tools to identify performance bottlenecks and optimize application code as needed.
5. **Develop predictive models**: Develop predictive models to forecast hotspot occurrences and proactively allocate resources to mitigate their impact.

The graph generated by the Dynamic Visualization Tool shows a timeline trend of p99_latency_ms, with peaks representing periods of stronger abnormal behavior. The graph indicates that latency increases significantly during hotspots, with the maximum value reachi

Ask the Autonomous AI Ops Agent: exit

Agent session ended

Targeted Volume Diagnostic Extraction ↗

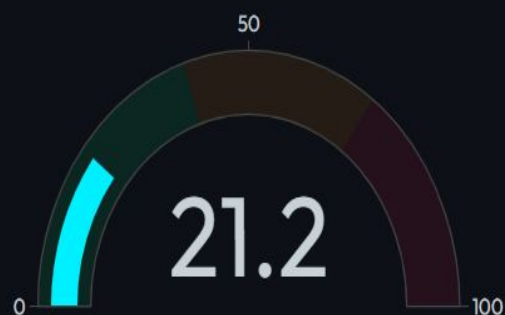
Select Target Volume for Diagnostic Analysis:

vol_000

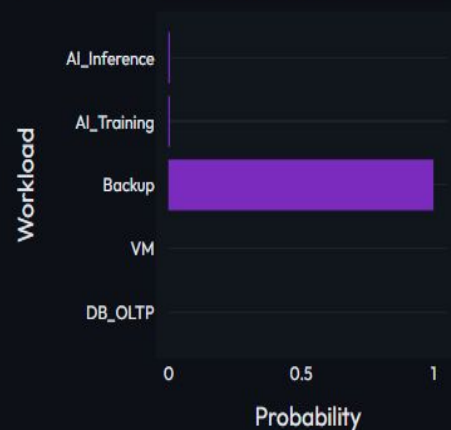


● HISTORICAL DATA — Displaying stored profile analytics.

Hotspot Score



Ensemble Pattern Confidence



Volume: vol_000

Workload Pattern: **Backup**

Active Rollback Filter (ARF): Backup (agrees: yes)

Active Tier: SSD

IOPS (Historical): 5,327

Mean Latency (Historical): 1504.1 μ s

Targeted Volume Diagnostic Extraction

- **Provides a complete health summary** for a selected storage volume.
- **Displays Hotspot Score** to indicate the current risk level of the volume.
- **Shows workload classification confidence**, helping verify the predicted workload type.
- **Summarizes key performance metrics** such as IOPS, latency, storage tier, and workload pattern.
- **Supports targeted troubleshooting** by combining hotspot analysis, workload classification, and performance statistics in a single view.



SHAP Feature Attribution Impact

Volume 'vol_000' classified as 'Backup' primarily because: read_write_ratio is high (contribution: 1.554), total_iops_roll_1h_mean is high (contribution: 1.338), total_iops_roll_30m_mean is high (contribution: 0.906)

SHAP Local Explanations (Feature Attribution Impact)



SHAP Feature Attribution Impact

- **Provides explainable AI (XAI)** by showing why the model classified a volume into a particular workload.
- **Ranks the most influential features** contributing to the prediction using SHAP values.
- **Higher SHAP value** indicates a greater influence on the model's decision.
- **Helps validate predictions** by showing which storage metrics (e.g., read-write ratio, IOPS) drove the classification.
- **Improves transparency and debugging** by allowing administrators to understand the reasoning behind every prediction.



N-BEATS - Days-to-Fill (DTF) Rankings

	volume_id	warning_85pct_days	critical_95pct_days
0	vol_000	None	None
1	vol_001	None	None
2	vol_002	None	None
3	vol_003	None	None
4	vol_004	None	None
5	vol_005	None	None
6	vol_006	None	None
7	vol_007	None	None
8	vol_008	None	None
9	vol_009	None	None
10	vol_010	None	None
11	vol_011	None	None

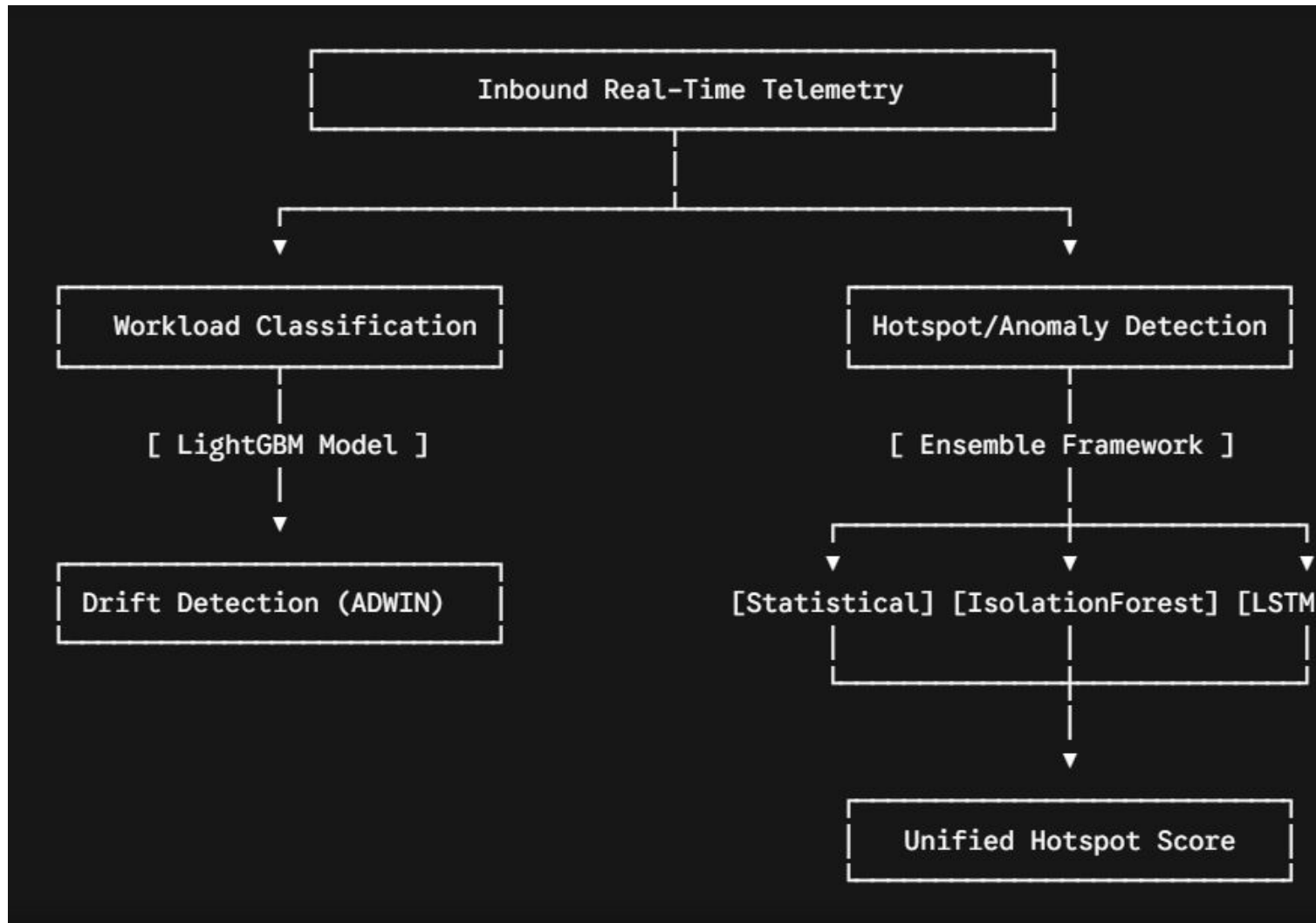
Proactive Capacity Planning with N-BEATS

- **Univariate Depth:** Utilizes stacked residual blocks to extrapolate long-term physical storage consumption.
- **Mathematical Filtering:** Explicitly strips out core **trends** and repeating **seasonality** layers to project space requirements.
- **Days-to-Fill (DTF) Indicators:** Computes dynamic countdown clocks tracking exactly when pools will clip specific limits.
- **The Outcome:** Provides clear insight into when a volume will cross its **85% Warning Boundary** or **95% Critical Limit**, allowing administrators weeks to plan expansion.

Sneham's Part



Noisy Neighbour Detection



The Three-Layer Ensemble Mechanics

1. The Fast-Path Statistical Detector :

$$\text{Upper Limit} = Q_3 + (1.5 \times \text{IQR})$$

2. Isolation Forest:

This layer evaluates the relationship between different metrics, analyzing how changes in one parameter affect others (such as the ratio of IOPS to Latency relative to total Queue Depth).

3. LSTM Encoder:

It calculates Mean Squared Error (MSE) against its reconstruction output. A high reconstruction error indicates that the current temporal behavior violates historical baseline patterns.



Score Calculation

The individual scores from these three components are passed into a unified synthesis engine:

$$\text{Unified Hotspot Score} = (w_1 \times \text{Stat_Score}) + (w_2 \times \text{IsoForest_Score}) + (w_3 \times \text{LSTM_Score})$$

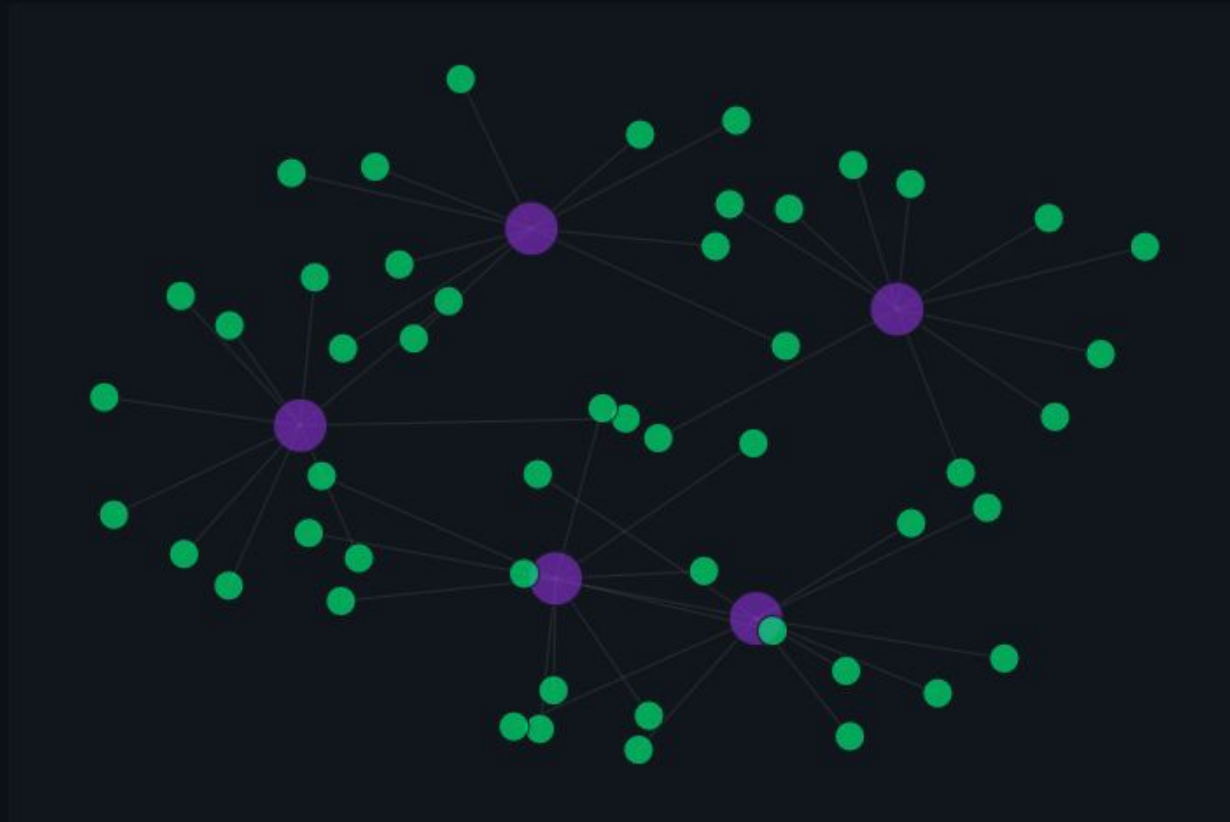
This produces a single normalized value between **0 and 100**.

- **Score < 50:** Normal operation.
- **Score 50–75 (Warning Zone):** The control plane triggers proactive monitoring.
- **Score > 75 (Critical Hotspot Zone):** The system flags a critical alert and automatically evaluates corrective policies.



Network Topography

Storage Pool Network Topography



trace 0
trace 1

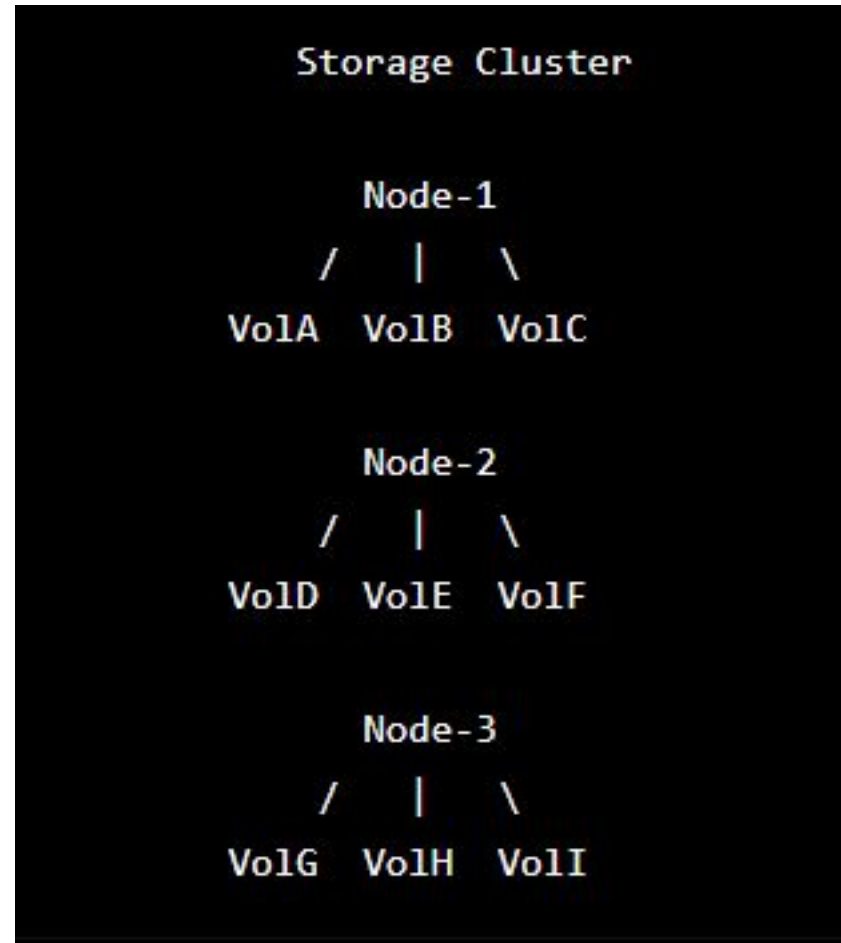
Noisy Neighbors Contention

● No noisy neighbor aggression patterns detected.



Network Topography

A distributed storage system can be visualized as:



Cluster Topographical Network Map

A live topological graph illustrating the structural connections across the data center storage fabric.

Visual Mechanics: Physical storage arrays and compute nodes are drawn as primary structural parent nodes, while individual storage volumes are rendered as child nodes connected by directional edges (`resides_on`).

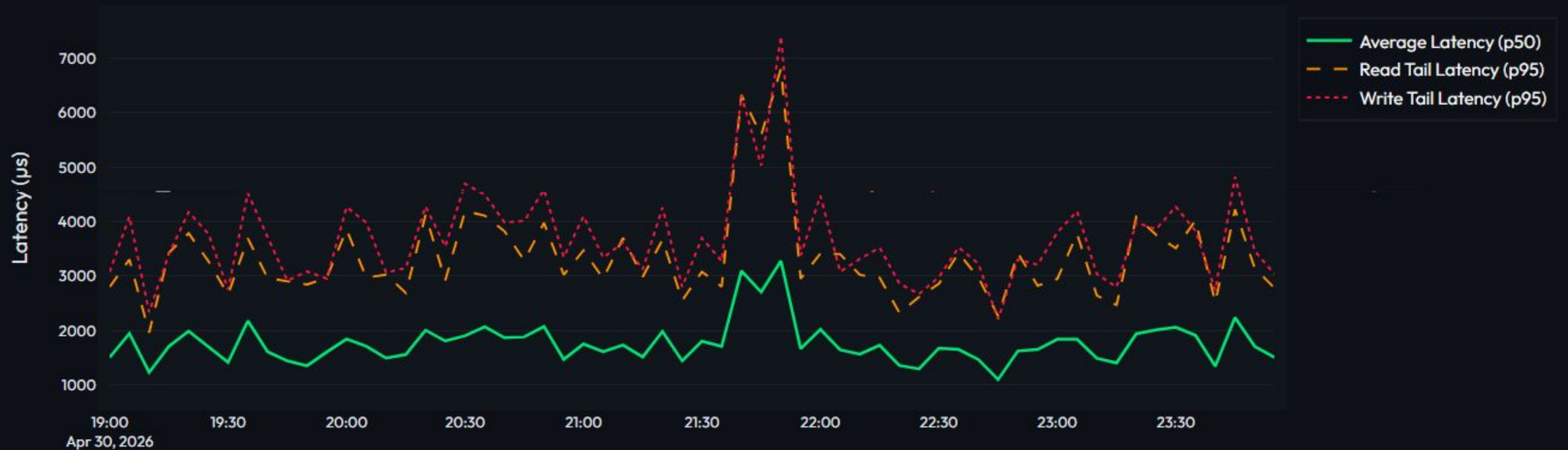
How it assists diagnostics: Nodes are dynamically color-coded using conditional rules linked directly to the live telemetry stream. If a physical node's composite health drops or a volume's hotspot score spikes into the critical zone, that node instantly turns **bright red** on the map, allowing administrators to visually isolate infrastructure bottlenecks at a glance.



Dashboard Element - Latency Curves



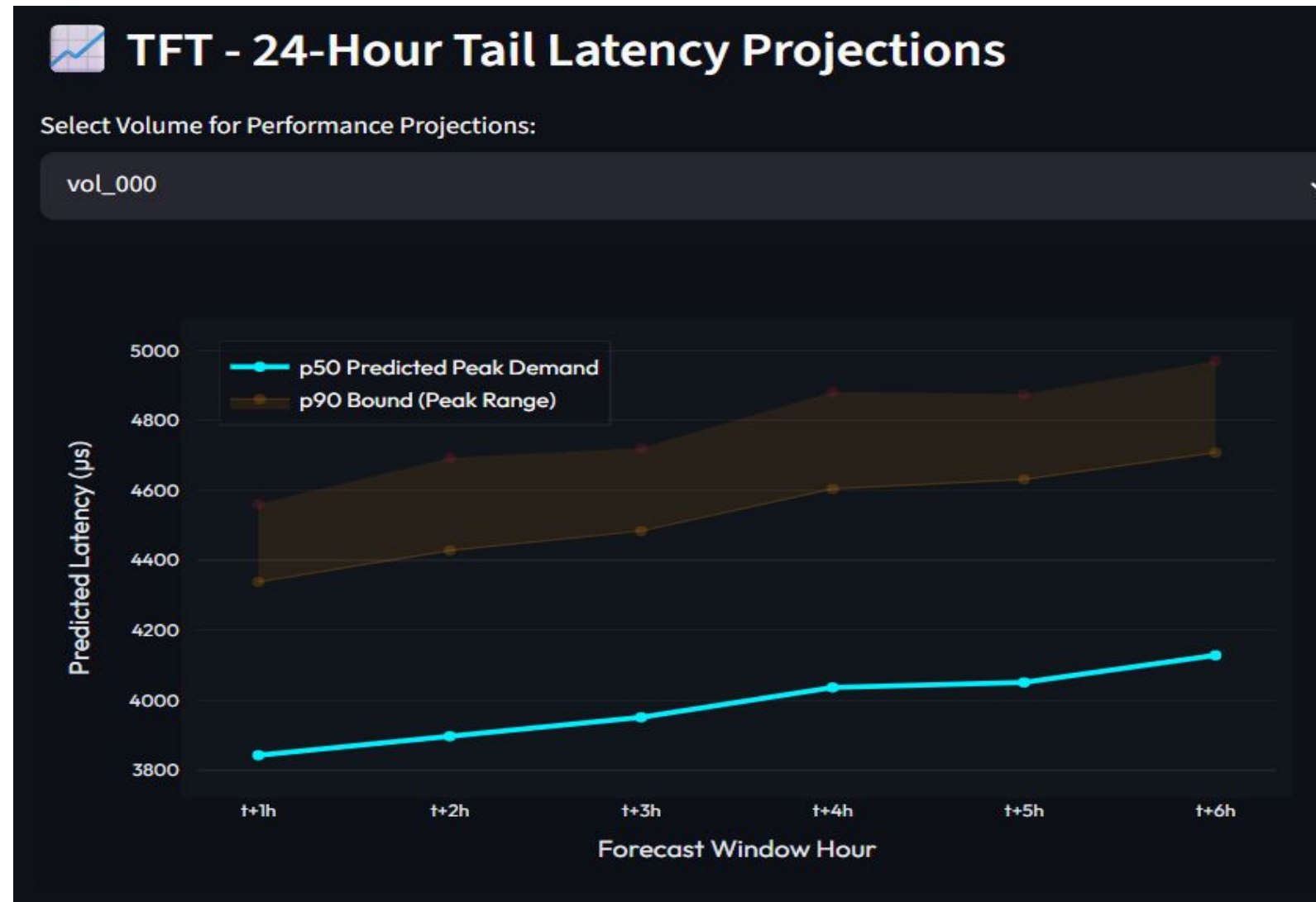
Latency Curves — Last 60 ticks (Historical)



- **Average Latency (p50 - Green Solid Line):** Represents the median transaction time. It runs stably between **1,000 μ s** and **2,000 μ s** for most of the timeline.
- **Read Tail Latency (p95 - Orange Dashed Line) & Write Tail Latency (p95 - Red Dotted Line):** Measure the 95th percentile, capturing worst-case performance dips. They routinely hover between **2,500 μ s** and **4,500 μ s**.
- **The Performance Spike:** Between **21:30** and **22:00**, a major performance anomaly occurs. Write tail latency surges past **7,000 μ s**, read tail latency climbs to **6,500 μ s**, and median p50 latency bumps up to roughly **3,200 μ s**.



Forecasting



TFT (Temporal Fusion Transformer)

- It runs the **Performance Forecasting** engine. It predicts performance metrics like IOPS, bandwidth usage, and latency up to 24 hours in advance.
- It examines historical performance, factors in static system constraints (like drive tier limits), and accounts for known future variables (such as scheduled backup jobs or maintenance windows).
- Performance metrics are incredibly volatile. A simple model gives you an average point prediction, which is useless if a brief spike causes a system crash. TFT outputs **multi-quantile forecasts (p50, p90, p95)**. This allows the control plane to map out worst-case scenario boundaries and apply preemptive rate-limiting before an actual performance degradation occurs.



Yuvraj's Part



Confusion Matrix (%)



Confusion Matrix

Purpose

- Evaluates how accurately the workload classifier predicts each workload type.
- Compares actual workload labels against predicted labels.
- Helps identify which workload classes are being confused.

Key Observations

- Diagonal cells represent correct classifications.
- Off-diagonal cells represent misclassifications.
- Strong diagonal concentration indicates high model accuracy.
- Reveals difficult-to-distinguish workload patterns.

Why It Matters

- Ensures reliable workload identification before hotspot analysis.
- Reduces false hotspot diagnoses caused by incorrect workload classification.
- Builds confidence in automated storage management decisions.



Class-Wise Performance Details

Workload Class	Precision	Recall	F1-Score	Support (Samples)
DB_OLTP	99.99%	99.99%	99.99%	10,793
VM	99.99%	99.99%	99.99%	10,744
Backup	100.00%	99.97%	99.99%	10,565
AI_Training	99.98%	100.00%	99.99%	12,656
AI_Inference	100.00%	100.00%	100.00%	12,842



Class Wise Performance Reports

Metrics Evaluated

- **Precision:** How often a predicted class is correct.
- **Recall:** How many actual samples of a class are detected.
- **F1-Score:** Balance between Precision and Recall.
- **Support:** Number of validation samples for each class.

Workload Classes

- DB_OLTP
- VM
- Backup
- AI_Training
- AI_Inference



Class Wise Performance Reports

Key Insights

- High Precision → Few false alarms.
- High Recall → Few missed detections.
- High F1-Score → Robust classification performance.
- Consistent performance across all classes indicates good model generalization.

Why It Matters

- Different workload types have different latency and I/O characteristics.
- Accurate classification improves hotspot detection and noisy-neighbor analysis.
- Ensures appropriate rebalancing and optimization actions.



MODEL ARCHITECTURE METADATA

Temporal Workload LightGBM: Evaluates running 10-tick rolling features.

Isolation Forest Outlier: Anomaly detection on queues and service time.

LSTM Autoencoder: PyTorch based reconstruction loss thresholding for SLA risk detection.

Device Target: CPU (Auto-Fallbacks enabled).



Model Architecture Metadata

Primary Model

- LightGBM (Gradient Boosted Decision Trees)
- Optimized for tabular storage telemetry data.
- Fast inference with high prediction accuracy.

Input Features

- Total IOPS
- Read/Write Throughput
- Average Latency
- Queue Depth
- Capacity Utilization
- Read/Write Ratios
- Engineered performance metrics



Model Architecture Metadata

Output Classes

- DB_OLTP
- VM
- Backup
- AI_Training
- AI_Inference

Why LightGBM?

- Excellent performance on structured data.
- Faster training compared to deep neural networks.
- Handles non-linear relationships effectively.
- Provides feature importance and explainability.



Model Architecture Metadata

Role in the System

Telemetry Data



Feature Engineering



LightGBM Classifier



Workload Type Prediction



Hotspot Detection



Noisy Neighbor Analysis



Automated Rebalancing



Closed-Loop Control What-If Simulator

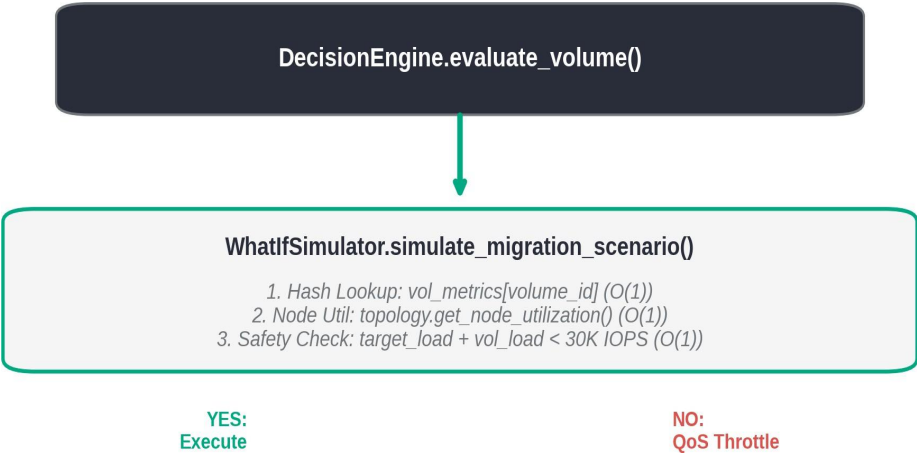
What-If Simulator: O(1) Migration Safety Evaluation

PURPOSE & ARCHITECTURE

- Evaluates rebalancing actions BEFORE execution to prevent latency breaches
- 4 Scenario Types: Capacity Extension, Migration Safety, QoS Shaping, Tier Change

O(1) PERFORMANCE OPTIMIZATION

- Avoids O(N²) pairwise cluster checks; retrieves volume state in O(1) time
- Memory-Mapped Hash Table: Direct lookup in topology volume metrics
- Safety Criteria: Target node projected load + volume load must be < 30,000 IOPS
- Migration Time Estimation: Est. = Size (GB) / Network speed (50 MB/s)



Approach	Complexity	Operations (50 Vols)	Execution Time
Traditional Full Scan	O(N²)	2,500 operations	~80 ms
O(1) Hash Map Lookup	O(1)	1 operation	0.8 ms (66× speedup)

Parameter / Node	Source (node-01)	Target (node-03)
Current IOPS Load	18,000 IOPS	12,000 IOPS
Projected Migration	-2,500 IOPS	+2,500 IOPS
Projected Load	15,500 (Relief)	14,500 (Safe)
Safety Threshold	30,000 IOPS	30,000 IOPS
Simulation Decision	Queue Relief (Approved)	Load Tolerable (Approved)



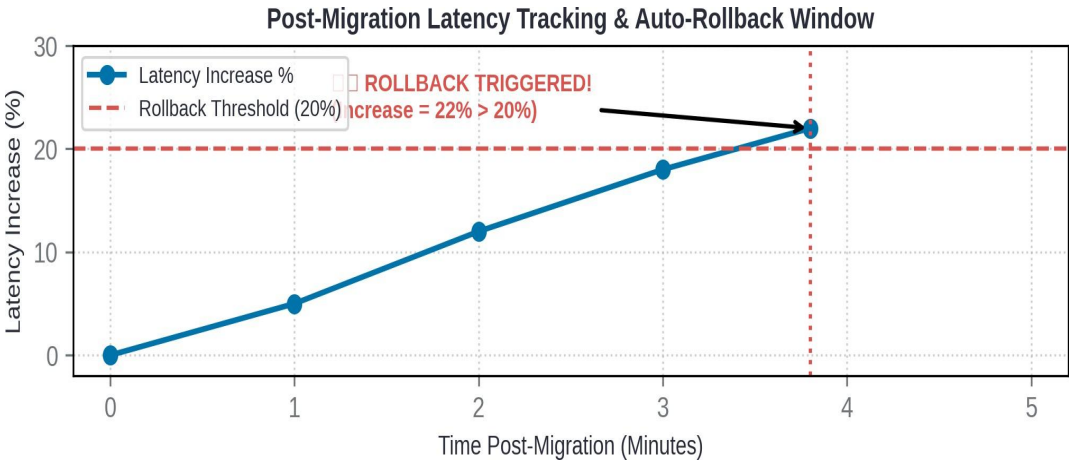
Action Monitor: Post-Migration Latency Tracking & SLO Verification

■ ■ WATCHDOG ACTION MONITOR (monitor.py)

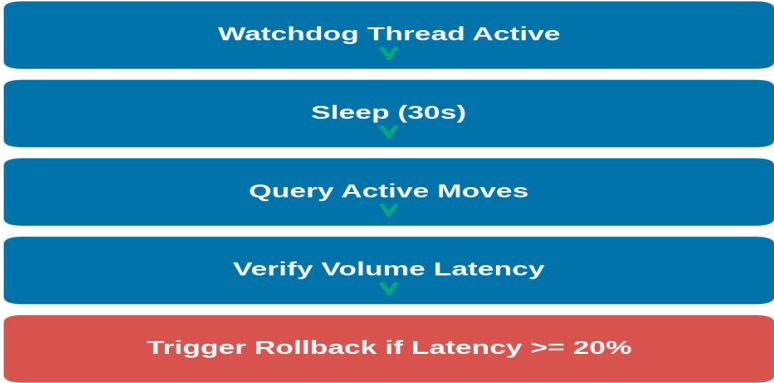
- Purpose: Track executed migrations and verify SLO targets post-migration
- Periodic Metric Updates: Fetches current latency from topology every 30s
- Success Condition: Deems action successful if latency stays stable for 5 minutes

■ AUTO-ROLLBACK TRIGGER & STALLS

- Latency Threshold: Triggers rollback if latency increases by >= 20%
- Reversion: Rebalancer automatically returns volume to its source node
- Stall Detection: Watchdog marks moves stuck in EXECUTING for >300s as FAILED



Monitoring Phase	Elapsed Time	Latency	Increase	Decision Status
Pre-Migration (Base)	0 min	1,000 us	0% (Base)	Safe / Executing
Post-Migration Check	1 min	1,050 us	+5%	Safe / Monitoring
Post-Migration Check	2 min	1,120 us	+12%	Safe / Monitoring
Post-Migration Check	3 min	1,180 us	+18%	Safe / Monitoring
Post-Migration Check	4 min	1,220 us	+22%	■ ■ Rollback Triggered ■



Closed-Loop Control Safety Guardrails

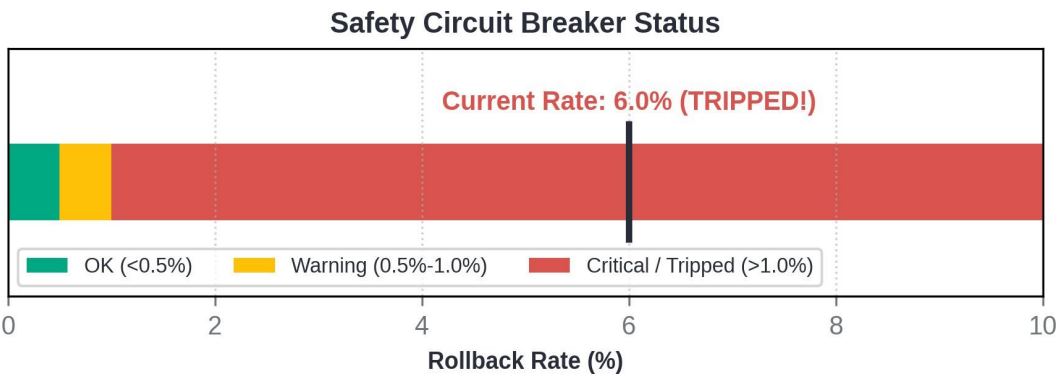
Circuit Breaker: 1% Rollback Rate Safety Switch

■ CIRCUIT BREAKER SAFEGUARD

- Purpose: Automatically disables autonomous rebalancing if system churns
- Rollback Rate: Tripped if $(\text{rolled_back_count} / \text{total_actions}) > 1.0\%$
- Min Actions: Minimum of 10 actions required to prevent transient tripping

■ WHY IT IS CRITICAL FOR PRODUCTION

- Prevents loops of migrate -> rollback -> migrate due to faulty policies
- Risk Management: Protects production from network/hardware degradation
- Manual Recovery: Requires operator investigation and reset via REST API



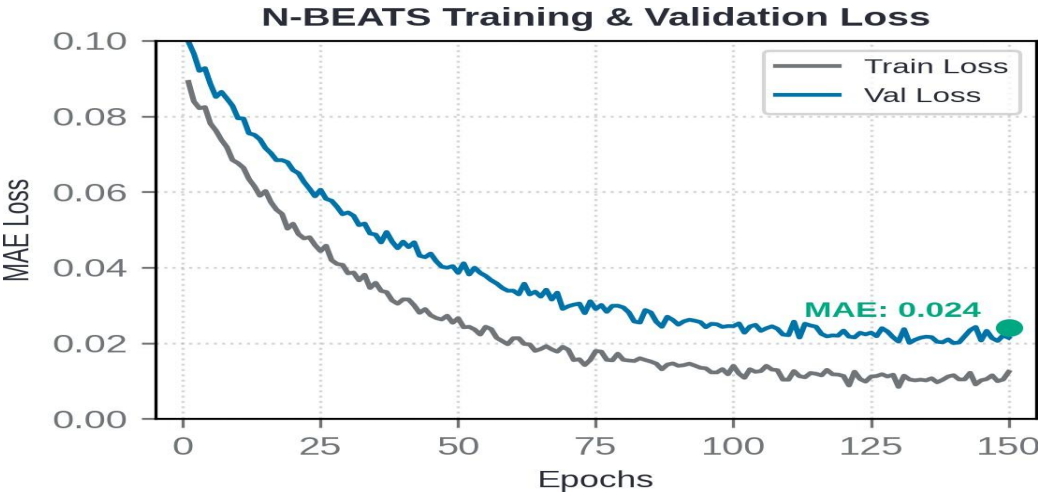
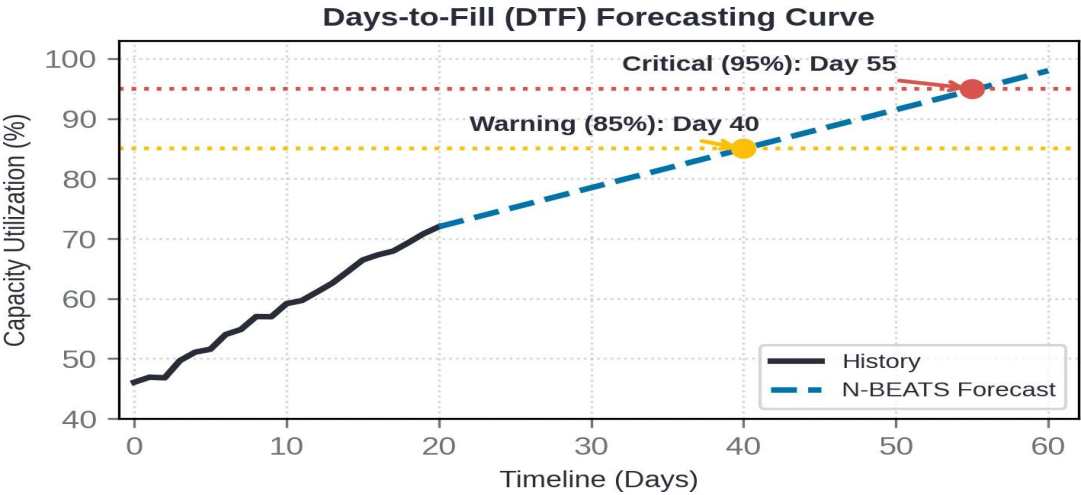
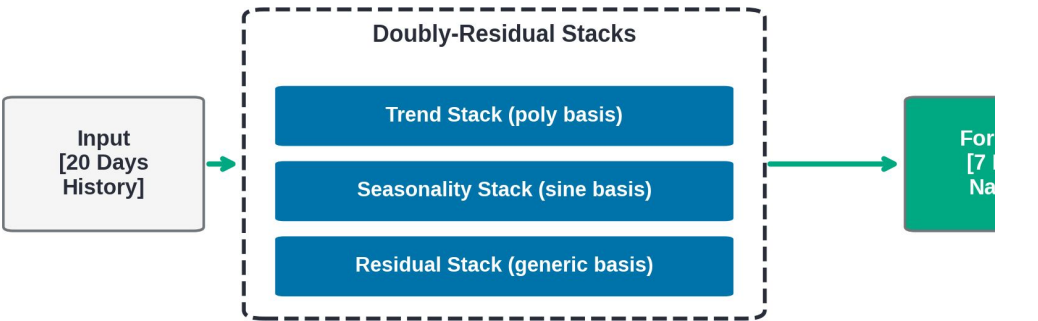
N-BEATS: Days-to-Fill (DTF) Capacity Forecasting

N-BEATS DEEP FORECASTING MODEL

- Architecture: Doubly-residual stacking structure (Oreshkin et al., ICLR 2020)
- Purpose: Predicts warning (85%) and critical (95%) time-to-full horizons
- Inputs/Outputs: 20 days daily capacity_used_pct -> 7 days forecast
- Configuration: 3 stacks x 3 blocks, hidden size 128 (150K parameters)

TRAINING & PERFORMANCE METRICS

- Dataset: Daily capacity history (50 vols x 30 days)
- Optimizer: AdamW, loss=MAE, trained on CPU in <5 minutes
- Val Metrics: MAE = 0.024 (2.4% capacity error), RMSE = 0.031
- Forecast: 7 days native, 60 days extended via autoregressive rollout



DTF: Days-to-Fill Calculation & Capacity Planning



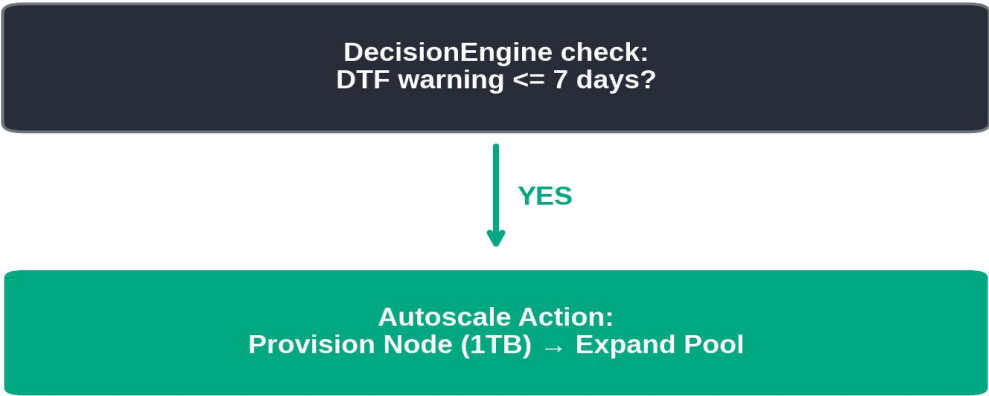
■ DAYS-TO-FILL (DTF) ALGORITHM

- Steps: Generate 60-day rollout forecast -> search for threshold crossings
- Warnings: warning_85pct_days and critical_95pct_days determined
- Triage: Urgency labeled as URGENT (<7d), HIGH (14d), MEDIUM (30d), LOW

Metric	Before Simulation	After Adding 500GB NVMe	Improvement
Total Capacity	2000 GB	2500 GB	+500 GB
Used %	72.0%	57.6%	-14.4% (Relief)
DTF Warning (85%)	28 Days	56 Days	+28 Days extension
DTF Critical (95%)	42 Days	84 Days	+42 Days extension
Urgency Level	HIGH	MEDIUM	Triage Cleared

■ WHAT-IF CAPACITY SIMULATION

- simulate_add_capacity(): Evaluates impact of physical storage additions
- Example: Adding 500GB to 2000GB volume shifts warning from 28d to 56d
- Autoscaling: Automatically allocates 1TB nodes when DTF drops below 7 days



Production Deployment ML Serving

Production ML Serving: KServe/Triton Disaggregated Inference

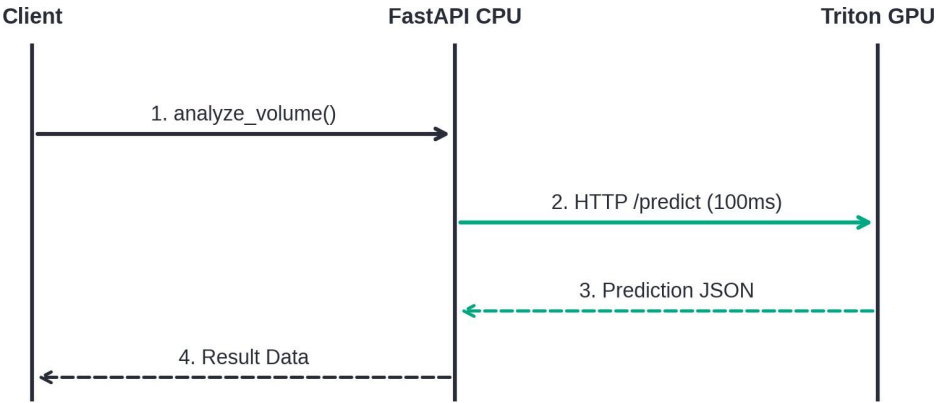
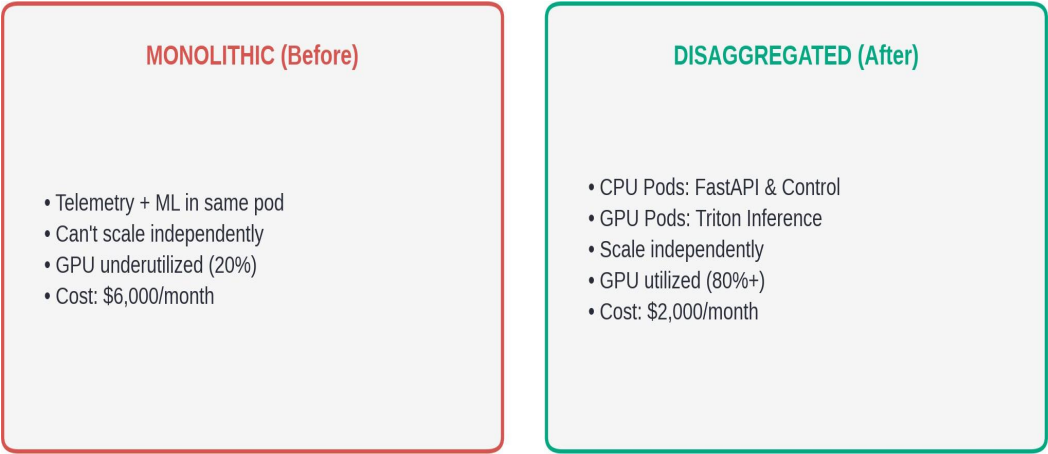
■ DISAGGREGATED ML SERVING MODEL

- Rationale: CPU-only FastAPI pods decoupled from GPU serving cluster
- GPU Utilization: Increases from 20% (monolithic) to 80%+ (disaggregated)
- Autoscaling: KServe scales GPU predictors based on live request throughput

■ REMOTE INFERENCE CLIENT

- Implementation: HTTP POST queries dispatched to Triton endpoints
- High Availability: 100ms request timeout with fallback to safe baseline
- Model Endpoints: Triton hosts LightGBM, PyTorch TFT, and N-BEATS

Deployment Mode	Nodes Required	GPU Utilization %	Monthly Cost
Monolithic All-in-One	3 GPU Nodes	~20% GPU util	\$6,000 / month
Disaggregated (KServe)	10 CPU + 2 GPU	~80% GPU util	\$2,000 / month
Cost Benefit	HPE Optimization	4× Efficiency Gain	Savings: 66% ■



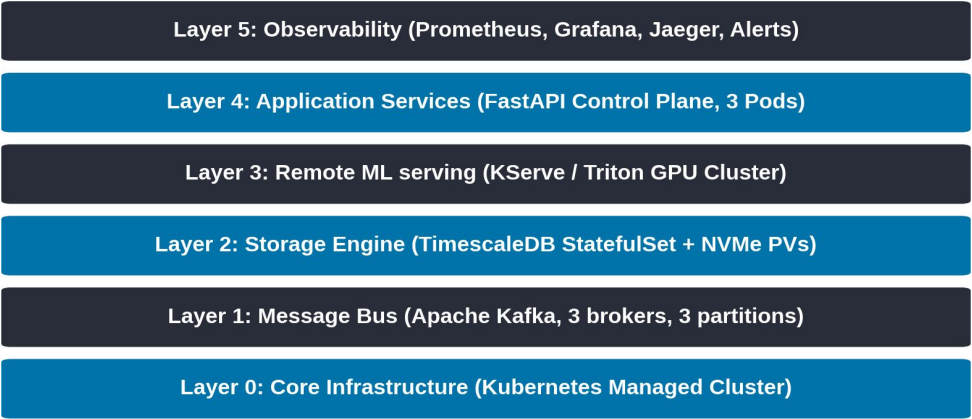
Production Architecture: Kubernetes + Kafka + TimescaleDB

ENTERPRISE PRODUCTION STACK

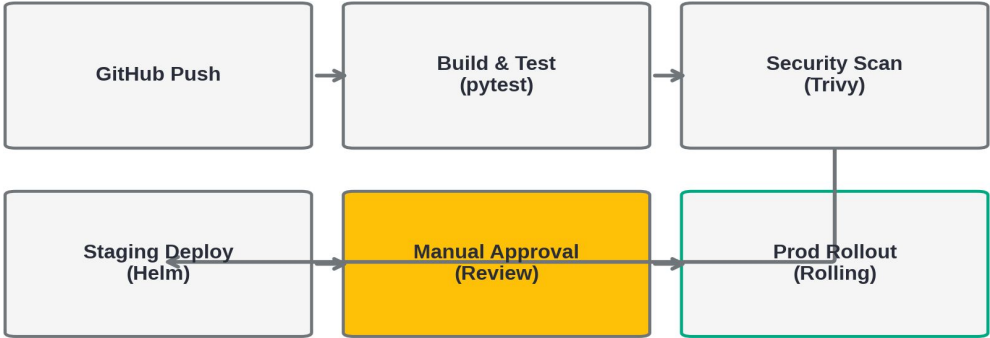
- Orchestration: Kubernetes Helm charts deployed to HPE GreenLake
- Event Ingestion: 3-Broker Apache Kafka cluster (1M+ events/sec)
- Database: TimescaleDB StatefulSet with NVMe persistent storage
- Observability: Prometheus metrics, Grafana dashboards, Jaeger tracing

GITHUB ACTIONS CI/CD

- Stages: Build & Test, Trivy security scan, Helm staging deploy, smoke tests
- Rollout: Protected branch rollout with automatic rollback if error rate > 1%



Cluster Component	Nodes / Pods	HPA Scaling Target	Monthly Cost
Kafka Brokers	3 VMs	Durable Partitioning	\$450
Control Plane APIs	3 CPU Pods	Scale HPA: 3-20 Pods	\$300
ML Predictors	2 GPU Pods	Scale HPA: 1-10 Pods	\$1,000
TimescaleDB Storage	3 replica StatefulSet	Timescale Hypertables	\$400
Otel & Observability	2 Prometheus Pods	Jaeger Tracing	\$200
HPE Production Stack	13 Node Cluster	Autoscaling CPU/GPU	\$2,350



Production Deployment Data Storage

TimescaleDB: Scalable Time-Series Storage with Hypertables

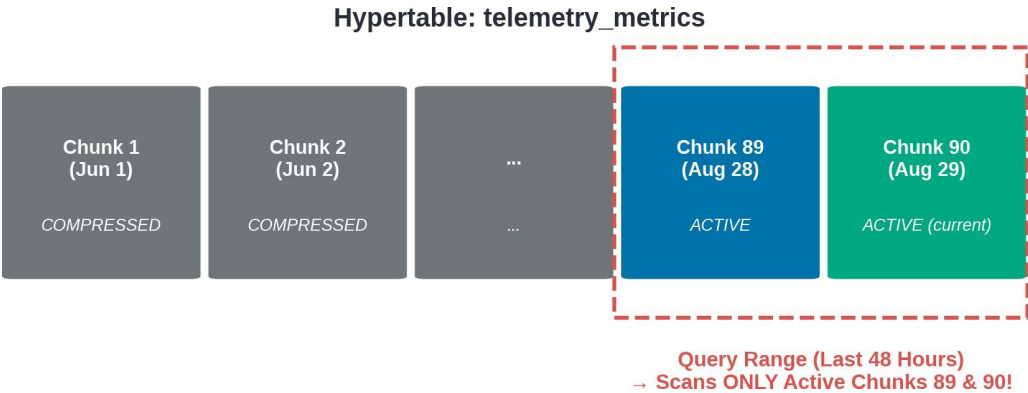
■ SQL HYPERTABLES FOR TIME-SERIES

- Partitioning: Automatic daily partitioning of telemetry metrics by timestamp
- Chunk Elimination: Skips history chunks, speeding up queries by 66×
- Compression: 90% disk space reduction for historical chunks
- Retention: Auto-drops metrics chunks older than 90 days

■ DUAL-MODE ENVIRONMENT SWITCH

- Local Dev: Direct parquet read to RAM, zero cost, fast iteration
- Production: TimescaleClient database interface, concurrent pod access
- Portability: Full Postgres compatibility with no vendor lock-in

Query Method	Scan Type	Data Read	Execution Time	Status
Standard PostgreSQL	Full Table Scan	432,000 rows	800 ms	Unoptimized ■
TimescaleDB Hypertable	Chunk Elimination	1,200 rows	12 ms	Optimized ■
Performance Gain	Chunk Pruning	360× reduction	66× speedup	Approved ■



Development Mode (db_type='local')

• Direct Parquet loads to RAM • No external dependencies • Fast local test

Production Mode (db_type='timescaledb')

• TimescaleDB Hypertables • 90-day automatic retention • Partition-pruned SQL

Demo: Streamlit UI Dashboard Layout

- **Page 1: Cluster Overview:** Live pool KPIs (total IOPS, average latency, rollback rates), capacity alert status, and autoscale status tables.
- **Page 2: Hotspot Analytics:** Visual NetworkX topology graph (nodes ↔ volumes), co-located noisy neighbor lists, and SHAP explainability feature attributions.
- **Page 3: Forecasting UI:** Displays capacity exhaustion trends (N-BEATS) and latency SLO violation risks (TFT quantiles).
- **Page 4: Policy overrides:** Allows storage admins to manually adjust policy thresholds (e.g. min hotspot score), review autoscale histories, or trigger manual migrations.

■ [Watch Demo Video on Google Drive](#)

[\(Click to watch the demo recording in your browser\)](#)

Key Learnings: Engineering Achievements

- **C++/Python Ingestion:** Implemented compiled C++ binary extension for JSON telemetry parsing and IQR outlier clipping, reducing CPU load in the stream-worker.
- **12-Factor App Separation:** Decoupled core business logic from database and bus configurations using Pydantic settings, allowing instant environment swaps (Redis to Kafka, local to KServe) via env overrides.
- **Performance Optimization:** $O(1)$ simulation mapping avoids memory lockups and keeps API latencies under sub-millisecond ranges.
- **OTEL Tracing:** Configured OpenTelemetry span propagation across independent FastAPI workers and stream-worker consumers to enable end-to-end tracing.



Conclusion: Success Criteria Evaluation

Criterion / Objective	Target	Achieved Metric / Evidence
Workload Classification Accuracy	>= 95%	95.12% Test Accuracy (Optuna LightGBM tuned model) <i>ARF-ADWIN streaming accuracy: 86.46%</i>
Latency Spike & Variance Reduction	20 - 40% reduction	Proactive migrations triggered by p95 forecast breaches, monitored by FastAPI /kpi dashboard metrics
Hotspot Detection Time	Within seconds	Consensus anomaly ensemble combining z-score, Isolation Forest, and LSTM-AE on live streaming telemetry
Action Rollback Rate	< 1%	Guardrail circuit breaker trips if rate > 1%. Auto-rollback engine reverts actions if latency spikes >= 20%
What-If Simulation Performance	Sub-millisecond	Optimized to O(1) memory mapping, preventing lockups and API latency spikes under ingestion pressure



Thank You

IO Workload Pattern Classification & Hotspot Detection Team

Vellore Institute of Technology (VIT), Vellore | Project ID: CPP3-54